

Java 25 — PEM Encodings of Cryptographic Objects (Preview)

UniqueClassName: PEMEncodings

JEP: 470

Java Version: 25

Year: 2025

Status: Preview

JEP 470 introduces standard Java APIs for **encoding and decoding cryptographic objects in PEM format**. PEM is the familiar text format used by TLS certificates, public/private keys, certificate revocation lists, and many security tools. Java 25 provides PEMEncoder and PEMDecoder APIs in java.security, reducing the need for custom Base64/header/footer handling or third-party PEM libraries. ([OpenJDK CVM](#))

1. Feature overview

Aspect	Details
Feature	PEM Encodings of Cryptographic Objects
UniqueClassName	PEMEncodings
JEP	JEP 470
Introduced	Java 25
Status	Preview
Main APIs	PEMEncoder, PEMDecoder, DEREncodable, PEMRecord
Package	java.security
Purpose	Encode/decode cryptographic objects using standard PEM textual representation
PEM examples	CERTIFICATE, PUBLIC KEY, PRIVATE KEY, ENCRYPTED PRIVATE KEY, X509 CRL
Encoding	DER/binary data → Base64 → PEM text
Decoding	PEM text → binary/cryptographic Java object
Encryption support	Password-based encryption for private keys
Major benefit	Standard JDK API instead of manually parsing PEM or relying on an external library
Important limitation	It is a preview API , so it is not yet a permanent Java SE API

--	--

The Java 25 API supports common objects including X509Certificate, X509CRL, PublicKey, PrivateKey, KeyPair, X509EncodedKeySpec, and PKCS8EncodedKeySpec. ([OpenJDK CVM](#))

2. Advantages and disadvantages

Category	Advantages	Disadvantages
Standardization	Provides a standard JDK API for PEM processing	Preview API can change before becoming final
Dependencies	Can eliminate the need for a third-party PEM library for supported cases	Existing applications may already depend on mature libraries
Certificate handling	Directly handles X.509 certificates and CRLs	Does not magically support every possible PEM object as a typed Java cryptographic class
Key handling	Supports public/private keys and key pairs	Private-key handling still requires careful security practices
Encryption	withEncryption(char[]) can produce encrypted private-key PEM	Password-based encryption is not a replacement for a proper secret-management system
Decoding	PEM can be converted directly into Java security objects	Invalid or unsupported PEM content can result in exceptions or generic representations
Interoperability	PEM is widely supported by TLS, OpenSSL, Kubernetes, cloud infrastructure, etc.	PEM is textual and generally larger than the underlying DER/binary representation
Developer experience	Avoids manually stripping - ----BEGIN ...----- / -----END ...----- and Base64 decoding	Developers still need to understand PEM type and cryptographic object semantics
Thread safety	PEMEncoder is immutable and thread-safe	Does not eliminate security concerns around key storage or memory handling
Security	Provides an API for encrypted private-key encoding	Encoding a private key as PEM without encryption can expose the key if the file/string is compromised

Integration	Useful when Java applications communicate with systems that already use PEM	Migration from custom/third-party code may require adaptation
Maintenance	Less custom parsing code to maintain	Preview APIs create potential future migration work
Performance	JDK implementation can avoid unnecessary application-level parsing logic	PEM Base64 encoding/decoding adds processing and size overhead compared with raw binary formats
Portability	Works with standard Java security APIs	Requires a Java version/API level supporting the preview feature

3. When to use it

Situation	Use PEM Encodings?	Why	Real-time example
TLS certificate management	Yes	Certificates commonly arrive as PEM	Java service loads server.crt
Public/private key exchange	Yes	PEM is a common interchange format	Service receives public.pem from another service
Kubernetes secrets containing certificates	Yes	Kubernetes TLS material commonly uses PEM	Java application reads a TLS certificate and private key
Mutual TLS (mTLS)	Yes	Client certificates and private keys frequently use PEM	Microservice authenticates to an API using a client certificate
Certificate rotation	Yes	New certificate/key material can be decoded without custom PEM parsing	Load a renewed certificate from a secret store
OpenSSL interoperability	Yes	OpenSSL commonly produces PEM files	Java imports certificate.pem generated by OpenSSL

Certificate authority tooling	Yes	CA certificates and CRLs are commonly represented as PEM	Java application reads a CA certificate bundle
Secret-management integration	Yes, carefully	Secret managers may return PEM text	Retrieve an encrypted private key from a secret store
Java-based security utility	Yes	Removes repetitive PEM parsing code	Internal certificate-management service
Temporary migration from custom PEM parser	Yes	Can simplify the implementation	Replace custom Base64/header parsing
New production system requiring stable API	With caution	Feature is preview	Consider waiting for a finalized API
Long-lived public library	Usually no	Preview API may change	Library intended to support many Java releases
Application requiring Java 8/11/17	No	Java 25 API isn't available there	Legacy enterprise application
Highly specialized PEM format	Maybe not	Typed support may not exist	Proprietary cryptographic PEM structure

4. When not to use it

Scenario	Why not?	Better approach
You must support Java 17	Java 25 preview API isn't available	Existing JCA/JCE APIs or compatible third-party library
Stable public API is required	Preview APIs can change or be removed	Wait for final API or abstract the implementation
You already have a mature PEM library	Migration may provide little immediate benefit	Keep the existing library if it meets requirements

You need a proprietary PEM type	Standard typed mapping may not exist	Generic PEM handling or specialized library
Private keys are being stored insecurely	PEM encoding doesn't secure the filesystem/secret	KMS/HSM/secret manager + encrypted private key
You need high-performance binary transport	PEM adds Base64/text overhead	DER or another binary representation
PEM is only an internal representation	There may be no need to convert to text	Keep cryptographic objects/binary form internally
Password is hard-coded	Encryption becomes ineffective	External secret management
You need hardware-backed private keys	PEM export may defeat the security model	HSM/PKCS#11/KeyStore-based approach

5. Real-time use cases with examples

Use case	Typical PEM	Example	Why PEM Encodings helps
HTTPS server certificate	CERTIFICATE	-----BEGIN CERTIFICATE-----	Decode certificate directly into X509Certificate
mTLS client	CERTIFICATE + PRIVATE KEY	Service-to-service authentication	Decode certificate and private key using JDK APIs
JWT signing key	PRIVATE KEY	RSA/EC/EdDSA signing key	Import a PEM private key into Java security APIs
JWT verification	PUBLIC KEY	API gateway verifies signatures	Decode public key directly
Kubernetes TLS secret	CERTIFICATE + PRIVATE KEY	tls.crt, tls.key	Avoid custom PEM parsing
Certificate rotation	CERTIFICATE	New certificate from CA	Decode replacement certificate
OpenSSL integration	Multiple PEM types	OpenSSL-generated	Easier Java

		keys/certificates	interoperability
CA trust management	CERTIFICATE	Root/intermediate CA files	Decode certificates into Java objects
CRL processing	X509 CRL	Revocation checking	Decode CRL into X509CRL
Encrypted private-key storage	ENCRYPTED PRIVATE KEY	Password-protected key	JDK can encode/decode encrypted private keys

6. Simple Java example — encode a private key

Java 25's PEMEncoder can encode a PrivateKey directly. ([OpenJDK CVM](#))

```
import java.security.KeyPair;
import java.security.KeyPairGenerator;
import java.security.PEMEncoder;

public class PemExample {

    public static void main(String[] args) throws Exception {

        KeyPairGenerator generator =
            KeyPairGenerator.getInstance("RSA");

        generator.initialize(2048);

        KeyPair keyPair = generator.generateKeyPair();

        PEMEncoder encoder = PEMEncoder.of();

        String pem = encoder.encodeToString(keyPair.getPrivate());

        System.out.println(pem);
    }
}
```

Conceptually, the output is:

```
-----BEGIN PRIVATE KEY-----
MIIEv...
...
-----END PRIVATE KEY-----
```

The important point is that the application doesn't need to manually:

1. call `getEncoded()`,
2. Base64 encode the bytes,
3. add the PEM header,
4. wrap lines,
5. add the PEM footer.

The JDK API performs the PEM encoding.

7. Decode PEM back into a Java key

`PEMDecoder` can decode `PRIVATE KEY` into a `PrivateKey`, and similarly supports public keys, certificates, CRLs, and encrypted private keys. ([OpenJDK CVM](#))

```
import java.security.PEMDecoder;
import java.security.PrivateKey;

public class PemDecodeExample {

    public static void main(String[] args) {

        String pem = ""
            -----BEGIN PRIVATE KEY-----
            ...
            -----END PRIVATE KEY-----
            "";

        PrivateKey privateKey =
            (PrivateKey) PEMDecoder.of().decode(pem);

        System.out.println(privateKey.getAlgorithm());
    }
}
```

8. Encrypted private key

One particularly useful capability is password-based encryption of private keys.

```
char[] password = "change-this".toCharArray();

PEMEncoder encoder =
    PEMEncoder.of().withEncryption(password);

String encryptedPem =
```

```
encoder.encodeToString(privateKey);
```

This produces an ENCRYPTED PRIVATE KEY representation. The Java 25 API documentation specifies that `withEncryption(char[])` configures the encoder for encrypted private-key output. ([OpenJDK CVM](#))

For decoding:

```
char[] password = "change-this".toCharArray();
```

```
PEMDecoder decoder =  
    PEMDecoder.of().withDecryption(password);
```

```
PrivateKey privateKey =  
    (PrivateKey) decoder.decode(encryptedPem);
```

Important: encrypting the PEM is not the same as securely managing the password. In production, don't hard-code the password; use a secret manager, KMS, HSM, or equivalent mechanism.

9. Before vs. Java 25 approach

Aspect	Traditional approach	Java 25 PEM API
Obtain encoded key	<code>key.getEncoded()</code>	Pass <code>PrivateKey</code> directly
Base64 encoding	Application/library code	JDK API
PEM header	Manually construct	JDK API
PEM footer	Manually construct	JDK API
Line wrapping	Manually/library	JDK API
Decode Base64	Manual/library	JDK API
Create <code>PrivateKey</code>	<code>KeyFactory</code> + <code>PKCS8EncodedKeySpec</code>	<code>PEMDecoder</code>
Certificate decoding	<code>CertificateFactory</code>	<code>PEMDecoder</code>
Encrypted private key	Additional implementation/library work	Supported by <code>PEMEncoder</code> / <code>PEMDecoder</code>
Third-party dependency	Often required	Potentially unnecessary

API stability	Existing APIs are mature	Preview
----------------------	--------------------------	----------------

10. Alternate features

Alternative	Type	When preferable	Comparison with PEMEncodings
JCA/JCE KeyFactory	Standard Java API	You already have DER/encoded key bytes	Lower-level; doesn't directly provide PEM handling
CertificateFactory	Standard Java API	You need to parse X.509 certificates	Excellent for certificates, but not a general PEM API
KeyStore	Standard Java API	Application needs managed key/certificate storage	Better for key storage than PEM files
PKCS#12 (.p12/.pfx)	Standard keystore format	You need certificates + private keys in a protected container	Better for managed key containers; less convenient for text-based interoperability
JKS	Java keystore	Legacy Java-centric key management	Java-specific and less interoperable than PEM
Bouncy Castle PEM APIs	Third-party	Need broad/custom cryptographic format support	Mature and feature-rich, but adds dependency
OpenSSL	External tool/library	Existing infrastructure is OpenSSL-based	Excellent ecosystem interoperability but outside the Java standard API
HSM / PKCS#11	Hardware-backed security	Private keys must never leave secure hardware	Much stronger key protection; PEM is generally not the right storage model
KMS/Secret Manager	Managed security service	Cloud/server-side production systems	Better for operational secret management than storing PEM directly

11. Most important distinction

A common misconception is:

PEM = encryption

It is **not**.

PEM is primarily a **textual encoding/transport format**. It takes binary cryptographic data, Base64-encodes it, and surrounds it with a type-specific header/footer. The Java API documentation describes PEM as Base64-formatted binary encoding enclosed by a type-identifying header and footer. ([OpenJDK CVM](#))

For example:

```
Binary DER
  ↓
Base64
  ↓
PEM header/footer
  ↓
-----BEGIN PUBLIC KEY-----
...
-----END PUBLIC KEY-----
```

Therefore:

Concept	PEM
Text representation	✓
Base64 encoding	✓
Interoperability format	✓
Automatically confidential	✗
Automatically integrity-protected	✗
Private-key encryption	Optional/supported
Secret management	✗
HSM protection	✗

12. Final recommendation

Requirement	Recommendation
Java 25 application needs to read/write standard PEM	Use PEMEncoder / PEMDecoder
mTLS certificates and keys	Use it
OpenSSL ↔ Java interoperability	Use it
Kubernetes TLS material	Use it
Certificate/key rotation	Use it
Need encrypted private-key PEM	Use it, with proper password/secret management
Long-lived public library	Be cautious because it is preview
Java 17 or earlier compatibility	Don't use it
HSM-backed keys	Prefer HSM/PKCS#11
Managed cloud secrets	Prefer KMS/Secret Manager
Java-native key storage	Consider KeyStore/PKCS#12
Need highly specialized PEM formats	Consider Bouncy Castle or another specialized implementation

Bottom line: PEMEncodings is primarily a **developer-productivity and interoperability feature**. Its biggest advantage is bringing common PEM processing into the JDK through PEMEncoder and PEMDecoder, rather than requiring every Java application to implement PEM parsing or depend on a third-party library. However, because it is a **preview API**, production teams should isolate its use behind their own abstraction if long-term API stability is important. ([OpenJDK CVM](#))

Java 25 — Stable Values (Preview)

UniqueClassName: StableValues

JEP: 502

Java Version: 25

Year: 2025

Status: Preview

Stable Values introduce a mechanism for storing a value that is **initialized at most once and then remains unchanged**. They are particularly useful for **lazy initialization**: the value can be computed when first needed, while allowing the JVM to optimize access to the value afterward.

Aspect	Details
Feature	Stable Values
JEP	502
Java version	25
Status	Preview
Main API	StableValue
Package	java.lang
Primary purpose	Lazily initialize a value exactly once and subsequently treat it as stable
Main problem addressed	Efficient, safe lazy initialization without repeatedly checking mutable state
Typical use	Lazily initialized configuration, lookup tables, compiled patterns, expensive immutable objects
Key property	Once initialized, the value cannot be changed
Thread safety	Designed for safe concurrent initialization/access
Relationship to final	Similar goal of immutability/stability, but supports delayed initialization
Relationship to volatile	Avoids repeatedly maintaining/accessing a mutable volatile state after initialization
Relationship to Supplier	Provides a stronger once-only/stable-value abstraction rather than recomputing on every call

1. Advantages and disadvantages

Area	Advantages	Disadvantages
Lazy initialization	Expensive objects can be created only when actually needed	Adds complexity when eager initialization would be sufficient
Immutability	Once initialized, the stored value cannot be replaced	The value itself must generally be designed to be immutable if you want deep immutability
Performance	JVM can exploit the stable nature of the value for optimization	Performance benefit depends on workload and JVM optimization
Thread safety	Designed for safe concurrent initialization	Developers still need to understand initialization semantics
Boilerplate	Can simplify some lazy-initialization patterns	For simple objects, final initialization is simpler
Memory	Avoids allocating an expensive object until needed	The stable holder itself introduces an additional abstraction/object
JIT optimization	Stability gives the JVM stronger information about the value	Optimization is JVM-dependent; don't assume a guaranteed speedup
Repeated access	Good for frequently accessed values after initialization	If the value is rarely accessed, the abstraction may not provide much benefit
Configuration	Useful for expensive configuration objects created on demand	Configuration that must dynamically change is unsuitable
Caching	Excellent for one-time initialization	Not a replacement for a cache whose entries need expiration/replacement
API design	Makes "initialize once, never change" semantics explicit	Preview API means source/binary compatibility can change
Debugging	Clear lifecycle:	Incorrect initialization logic can be

	uninitialized → initialized	harder to diagnose
Failure handling	Can defer expensive initialization until required	Initialization failures need deliberate handling depending on the API usage pattern

2. StableValue vs final

The biggest conceptual difference is **when** the value becomes assigned.

Feature	final field	StableValue
Assignment	During construction/initialization	Later, on first use
Can be lazy?	Not naturally	Yes
Can change after assignment?	No	No
Initialization time	Usually eager	Potentially deferred
Thread-safe lazy initialization	Requires additional pattern	Designed for this purpose
JVM knows it is stable	final provides strong semantics	Stable Values provide stability information
Typical use	Required object state	Expensive optional/on-demand state
Example	private final String name;	Lazily initialized lookup/configuration object

Example:

```
class UserService {
    private final UserRepository repository;

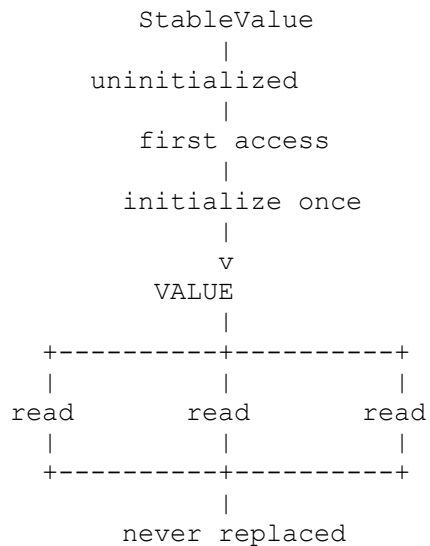
    UserService(UserRepository repository) {
        this.repository = repository;
    }
}
```

The repository is initialized when UserService is created.

With a stable value, an expensive object can instead be initialized only when needed.

3. Basic conceptual example

A Stable Value can be thought of conceptually as:



The key idea is:

"Compute it once, then keep that value forever."

4. When to use Stable Values

Situation	Use?	Reason
Expensive object is rarely needed	✓ Yes	Avoid unnecessary initialization
Expensive immutable configuration	✓ Yes	Initialize on demand
Frequently accessed derived value	✓ Yes	One-time initialization can avoid repeated computation
Compiled regular expression/pattern	✓ Yes	Compile only when required

Large lookup table	✓ Yes	Avoid startup cost
Expensive parser	✓ Yes	Construct on first use
Immutable metadata	✓ Yes	Initialize once
Service-wide constant calculated dynamically	✓ Yes	Value can be computed lazily
Application starts many components but uses only some	✓ Yes	Reduces unnecessary startup work
Value must change at runtime	✗ No	Stable values are intended to remain stable
Cache needs eviction	✗ No	Use a cache
Frequently changing configuration	✗ No	Use mutable configuration mechanism
Simple constant	✗ Usually	Use static final
Cheap object	✗ Usually	Eager initialization is simpler
Per-request state	✗ No	Use local/request-scoped state
Database data that changes	✗ No	Use repository/cache/database access

5. Real-time use cases

Use Case 1 — Expensive configuration object

Suppose an application builds a complicated immutable configuration:

```
class ApplicationConfig {
    // Expensive configuration construction
}
```

If only certain deployments actually need it, lazy initialization is useful.

Without Stable Value	With Stable Value
----------------------	-------------------

Configuration may be constructed during startup	Constructed only when required
Startup work occurs even if unused	Startup work can be deferred
Custom lazy-init code may be required	Stable-value abstraction expresses the intent
More synchronization/visibility concerns in custom implementations	API provides appropriate concurrency semantics

Real-world example:

A financial application has multiple pricing engines, but only one pricing model is selected for a particular deployment. Expensive pricing configuration can be initialized only when that engine is first accessed.

6. Use Case 2 — Compiled regular expression

Suppose a service repeatedly uses an expensive pattern:

```
Pattern.compile(
    "complex-expression..."
);
```

Instead of compiling it every time:

```
Request 1 → compile pattern → use
Request 2 → compile pattern → use
Request 3 → compile pattern → use
```

a stable value can conceptually provide:

```
First request
  ↓
compile pattern
  ↓
store stable value
  ↓
use

Later requests
  ↓
reuse same pattern
```

This is especially useful when the pattern is expensive to create and immutable afterward.

7. Use Case 3 — Large lookup table

Imagine:

```
Map<String, Integer> countryCodes;
```

Building the map may require substantial processing.

A stable value is appropriate when:

- the table is expensive to construct,
- it is immutable after construction,
- many operations reuse it,
- and it doesn't need to be refreshed.

Examples:

- ISO country-code lookup
 - currency metadata
 - HTTP status metadata
 - MIME-type lookup
 - static product classification
 - protocol constants
-

8. Use Case 4 — Expensive parser

A server might support an expensive parser:

Application starts

↓

Parser not needed yet

↓

Do NOT construct parser

↓

First request requiring parser

↓

Construct parser

↓

Reuse parser

This can reduce startup work for applications where some features are rarely used.

9. Use Case 5 — Framework metadata

Frameworks frequently have expensive metadata calculations:

Class
↓
Reflection / metadata analysis
↓
Build metadata model
↓
Reuse metadata

If the metadata is immutable once calculated, Stable Values are a natural candidate.

Examples:

- ORM entity metadata
- serialization metadata
- dependency-injection metadata
- HTTP endpoint metadata
- annotation processing results

10. When NOT to use Stable Values

Requirement	Why Stable Values are inappropriate	Better alternative
Value changes every request	Stable Values don't represent mutable state	Local variable/request context
Configuration reloads dynamically	Value needs replacement	volatile, AtomicReference, configuration manager
Cache entries expire	Value needs eviction/replacement	ConcurrentHashMap + cache policy
Need LRU caching	Multiple values and eviction are required	Caffeine/cache implementation
Simple constant	Stable Value adds unnecessary complexity	static final
Cheap initialization	Lazy initialization doesn't justify complexity	final / static final
Object depends on request	Lifetime is incorrect	Request-scoped object
Database-backed value	Data can become stale	Database/repository/cache

Mutable object is required	Stability of the reference doesn't make its contents immutable	Appropriate mutable abstraction
Java version must remain below 25	API unavailable	Traditional lazy initialization

11. Stable Value vs common alternatives

Alternative	Best for	Stable Values better when	Alternative better when
static final	Simple constants/eager immutable objects	Initialization is expensive and should be deferred	Value is cheap and always needed
final instance field	Immutable object state	Initialization should happen later	Object is naturally constructed with its owner
volatile	Mutable shared state / visibility	Value is initialized once and never changed	Value may change
AtomicReference	Atomic mutable reference	Reference should never change after initialization	Updates/swaps are required
Double-checked locking	Lazy singleton/object initialization	You want a standard abstraction for stable lazy initialization	Legacy compatibility or special custom semantics
Initialization-on-demand holder idiom	Lazy static singleton	You need more general stable-value semantics	Simple static singleton is all you need
Supplier<T>	Deferred computation	Computation must happen once and become stable	Recalculation is intentional
ConcurrentHashMap	Concurrent lookup/cache	Only one permanent value is needed	Multiple dynamic values are required
Caffeine/cache	Expiring/dynamic caching	Value is initialized once forever	Entries must expire or be evicted

Optional	Presence/absence modeling	Lazy stable initialization is the problem	You only need null/presence semantics
Dependency injection singleton	Application-wide service	Service construction should be lazy and stable	DI container already handles lifecycle

12. Stable Value vs Supplier

This is an especially important distinction.

Supplier

A supplier represents:

"Give me the value when I ask."

Depending on implementation, it may calculate repeatedly.

```
Supplier<Config> supplier =
    () -> createConfig();
```

Potentially:

```
get() → create
get() → create
get() → create
```

Stable Value

The desired semantic is:

```
first access
↓
create
↓
remember
↓
same value forever
```

Characteristic	Supplier<T>	StableValue<T>
Lazy	✓	✓

Naturally one-time	✗	✓
Stable after initialization	Not guaranteed	✓
Intended for mutable/recomputable computation	✓	✗
Expresses permanent stability	✗	✓
Good for memoization	Requires additional implementation	✓

13. Stable Value vs traditional double-checked locking

Traditional lazy initialization might look like:

```
private volatile Config config;
```

```
Config getConfig() {
    Config result = config;

    if (result == null) {
        synchronized (this) {
            result = config;

            if (result == null) {
                result = createConfig();
                config = result;
            }
        }
    }

    return result;
}
```

This works, but it is relatively complicated.

Conceptually:

Traditional approach

```
↓
volatile
↓
null check
```

↓
synchronized
↓
second null check
↓
initialize

Stable Values aim to provide a more direct abstraction:

Stable Value
↓
initialize once
↓
stable forever

The important benefit is not simply "fewer lines of code"; it is that the **stability semantics are explicitly represented**, giving the JVM an opportunity to optimize accordingly.

14. Performance perspective

Performance characteristic	Impact
Startup time	Potentially improved by deferring expensive initialization
First access	May be slower because initialization happens at that point
Subsequent access	Potentially very efficient
Memory	Avoids creating unused expensive objects
JIT optimization	Stable semantics can provide optimization opportunities
Throughput	Potentially improved for heavily reused stable values
Latency	First-use latency can increase
Predictability	Eager initialization may be preferable where first-request latency matters

Important trade-off

Stable Values essentially allow you to move work:

Startup



expensive initialization

to:

First actual use



expensive initialization

Therefore, **startup performance can improve while first-use latency can increase.**

For latency-sensitive applications, you should benchmark whether eager initialization or lazy initialization is preferable.

15. Practical decision table

Question	Answer	Recommendation
Is initialization expensive?	Yes	Consider Stable Value
Is the value rarely used?	Yes	Strong candidate
Is the value used frequently after first access?	Yes	Strong candidate
Will the value ever need replacement?	Yes	Don't use
Is the value cheap to create?	Yes	Prefer final/static final
Does the application require dynamic configuration?	Yes	Don't use
Is first-use latency critical?	Yes	Consider eager initialization
Is startup latency critical?	Yes	Stable Value may help
Is Java 25+ acceptable?	Yes	Consider it
Must the code run on	Yes	Don't use it

older Java versions?		
Is this a public library?	Yes	Be cautious while the API is preview
Is the object immutable?	Yes	Excellent candidate
Is the object internally mutable?	Yes	Carefully evaluate semantics

16. Alternate feature — recommendation hierarchy

Requirement	Preferred feature
Simple constant	static final
Immutable object initialized during construction	final
Lazy, one-time, permanently stable value	Stable Values
Lazy singleton	Initialization-on-demand holder idiom / Stable Value
Recomputable deferred operation	Supplier<T>
Mutable shared reference	volatile / AtomicReference
Concurrent collection/cache	ConcurrentHashMap
Expiring/evicting cache	Caffeine or equivalent cache
Java-native persistent key/value storage	KeyStore or appropriate persistence
Dynamic configuration	Configuration management system

17. Overall assessment

Category	Rating	Explanation
Developer productivity	★ ★ ★ ★ ★	Simplifies a difficult lazy-initialization problem
Readability	★ ★ ★ ★ ★	Clearly communicates "initialize

		once, then stable"
Performance potential	★ ★ ★ ★ ☆	Can reduce startup work and enable JVM optimization
Memory efficiency	★ ★ ★ ★ ☆	Avoids constructing unused expensive objects
Concurrency	★ ★ ★ ★ ★	Designed specifically for safe stable initialization
Flexibility	★ ★ ★ ☆ ☆	Intentionally unsuitable for values that need replacement
Simplicity for basic constants	★ ★ ☆ ☆ ☆	final/static final is simpler
Production readiness	★ ★ ★ ☆ ☆	Preview status requires caution
Long-term API stability	★ ★ ★ ☆ ☆	Preview API may evolve
Usefulness for frameworks	★ ★ ★ ★ ★	Particularly attractive for lazy metadata and infrastructure
Usefulness for ordinary business objects	★ ★ ★ ☆ ☆	Only worthwhile when lazy initialization provides a real benefit

Bottom line

Stable Values are most valuable when all three conditions are true:

Expensive to create + not needed immediately + never needs to change.

For example:

Expensive immutable configuration

+

lazy initialization

+

concurrent access

↓

Stable Value

If the value is cheap and always needed, use final/static final. If it changes, use a mutable/concurrent abstraction. If it is a real cache with expiration or eviction, use a cache rather than a Stable Value.

Because **Stable Values** is a **Java 25 preview feature**, production adoption should also account for preview-feature compilation/runtime requirements and the possibility that the API may change before finalization.

Java 25 — Remove 32-bit x86 Port

UniqueClassName: RemoveX86Port

JEP: 503

Java Version: 25

Year: 2025

Status: Final

JEP 503 removes the **32-bit x86 (x86-32 / i686) port from OpenJDK**. This means Java 25 no longer provides an official OpenJDK port for running the JVM on 32-bit x86 operating systems. The change is primarily a **platform-maintenance and simplification improvement**, rather than a new programming-language feature.

1. Feature overview

Aspect	Details
Feature	Remove 32-bit x86 Port
UniqueClassName	RemoveX86Port
JEP	503
Introduced	Java 25
Year	2025
Status	Final
Removed platform	32-bit x86 / x86-32
Also known as	i386/i486/i586/i686 class of 32-bit x86 systems
What remains	64-bit x86 (x86-64 / AMD64)

Primary reason	Reduce maintenance burden for an increasingly obsolete platform
Main beneficiaries	OpenJDK maintainers and users of modern 64-bit systems
Application source compatibility	Java source code itself is generally unaffected
Runtime compatibility	A Java 25 JVM cannot be used as the official OpenJDK runtime on a 32-bit x86 platform
Migration direction	Move to 64-bit x86, ARM64, or another supported architecture

2. Advantages

Advantage	Explanation	Real-world benefit
Reduced OpenJDK maintenance	Developers no longer need to maintain 32-bit x86-specific code	More engineering effort can go toward current platforms
Simpler JVM implementation	Removes architecture-specific 32-bit code paths	Less platform-specific complexity
Better focus on modern hardware	32-bit x86 hardware is increasingly uncommon	Resources can focus on 64-bit architectures
Reduced testing burden	Fewer architecture/OS combinations need testing	Faster and simpler OpenJDK testing
Modern memory model assumptions	Modern applications commonly require more memory than 32-bit address spaces comfortably provide	Better fit for large Java applications
Cleaner build infrastructure	Fewer platform-specific build configurations	Easier OpenJDK maintenance
Potentially faster development	Changes don't need to preserve compatibility with obsolete architecture-specific behavior	New JVM work can focus on modern targets
Better alignment with industry hardware	Most current servers, desktops and cloud instances are 64-bit	Java development aligns with current infrastructure

Security/maintenance efficiency	Avoiding obsolete platform support reduces the amount of legacy code that must be maintained	Simplifies long-term platform support
Reduced CI complexity	Fewer specialized builds and test environments	Lower infrastructure and maintenance cost

3. Disadvantages

Disadvantage	Explanation	Impact
32-bit x86 systems cannot use official OpenJDK Java 25	The official port has been removed	Existing systems must migrate or remain on an older Java release
Legacy hardware support ends	Old PCs, embedded systems and industrial machines may still use 32-bit x86	Hardware replacement may be necessary
Migration cost	Organizations may need to upgrade OS/hardware	Additional operational expense
Older operating systems	Some old operating systems are tied to 32-bit environments	Java 25 adoption may be blocked
Legacy applications	Existing Java applications may depend on 32-bit native libraries	JNI/JNA components may require migration
Third-party dependencies	Native libraries may only have 32-bit binaries	Libraries need 64-bit versions
Embedded environments	Some older embedded systems have limited hardware options	Upgrade can be difficult
Long-lived enterprise systems	Industrial applications can run unchanged for many years	Java upgrade becomes more complicated
Testing effort	Migration from 32-bit to 64-bit can expose	Regression testing is required

	assumptions about primitive sizes, native pointers, serialization, or JNI	
No benefit for the legacy user	Removing a port doesn't directly make an old application faster	Main benefits are platform maintenance and modernization

4. When to use Java 25 with this change

The important distinction is that **you don't explicitly "use" Remove 32-bit x86 Port in your Java code.**

You choose a **supported 64-bit Java 25 runtime.**

Situation	Recommendation	Reason
64-bit x86 server	✓ Use Java 25	Supported architecture
Modern Intel/AMD server	✓ Use Java 25	64-bit x86 is supported
Modern x86-64 desktop	✓ Use Java 25	Supported
ARM64 server	✓ Use appropriate Java 25 build	Modern supported architecture
Cloud VM using x86-64	✓ Use Java 25	Standard modern deployment
Container running on 64-bit Linux	✓ Use Java 25	Normal modern deployment
New application	✓ Prefer Java 25 where appropriate	Avoid introducing 32-bit dependencies
Enterprise application modernization	✓ Consider Java 25	Good opportunity to leave legacy 32-bit infrastructure
CI/CD infrastructure	✓ Use 64-bit Java 25 agents	Aligns with supported runtime platforms
Microservices	✓ Use Java 25 on 64-bit infrastructure	Modern deployment model

5. When NOT to use Java 25

Situation	Recommendation	Why
Application must run on a 32-bit x86 OS	✗ Don't use Java 25 OpenJDK	Official 32-bit x86 port has been removed
Legacy industrial PC is 32-bit only	✗ Don't upgrade blindly	Hardware/OS may not support Java 25
Critical application depends on 32-bit JNI libraries	✗ Don't migrate without testing	Native libraries may fail on 64-bit
Third-party vendor only provides 32-bit native binaries	✗ Don't migrate until vendor support exists	Native compatibility problem
Hardware cannot be upgraded	✗ Don't force Java 25	Platform constraint remains
Application has no requirement for newer Java	☐ Consider staying on a supported older release	Migration may not provide enough business value
Old embedded x86 device	☐ Evaluate carefully	Java 25 may no longer be a viable runtime
Legacy OS is 32-bit only	✗ Don't use Java 25 on that environment	Runtime/platform incompatibility

6. Real-time use cases

Use Case 1 — Enterprise server migration

Old environment

Old server
↓
32-bit x86 OS
↓
Java 8
↓
Enterprise application

Java 25 cannot simply be installed as the official OpenJDK runtime on that 32-bit x86 environment.

Modernized environment

New server
↓
64-bit x86 OS
↓
Java 25
↓
Enterprise application

Area	Old	New
CPU	32-bit x86	64-bit x86
JVM	Older Java	Java 25
Address space	Limited	Much larger
Platform support	Legacy	Modern
Long-term maintainability	Lower	Higher

7. Use Case 2 — Docker/container deployment

Suppose an organization has:

Java application
↓
Docker image
↓
Linux
↓
x86-64 server

There is **no problem caused by JEP 503**, because x86-64 is a 64-bit architecture.

The distinction is:

32-bit x86	✗ removed
x86-64	✓ supported

This is particularly relevant when choosing a Java container image: ensure the underlying image and deployment architecture are 64-bit.

8. Use Case 3 — Legacy industrial machine

Consider an industrial control system running:

Old industrial PC

↓

32-bit x86 CPU

↓

32-bit Windows/Linux

↓

Java application

↓

Machine-control software

This is a situation where you **should not blindly move to Java 25**.

The organization may need:

1. new hardware,
2. a 64-bit OS,
3. 64-bit native libraries,
4. application testing,
5. vendor certification.

For safety-critical or industrial systems, the migration should be treated as an infrastructure project rather than simply a Java version upgrade.

9. Use Case 4 — JNI application

Consider:

```
native int processDevice();
```

with a native library:

device.dll

If the existing library is only:

32-bit device.dll

moving the Java process to a 64-bit JVM generally requires a compatible:

64-bit device.dll

Therefore:

Component	32-bit	64-bit
JVM	32-bit	64-bit
Native library	32-bit	64-bit
OS	32-bit	64-bit
Architecture	x86	x86-64

All components need to be compatible.

10. Use Case 5 — CI/CD modernization

An organization might have old build agents:

Build Agent
↓
32-bit OS
↓
32-bit Java
↓
Maven/Gradle

A modern pipeline can instead use:

Build Agent
↓
64-bit Linux/Windows
↓
Java 25
↓
Maven/Gradle
↓
Application

This is a good example where JEP 503 encourages modernization without requiring changes to Java source code.

11. Impact on Java application code

This is an important point.

JEP 503 does not mean that Java int or long types have been removed or changed.

Java concept	Impact
byte	✗ No change
short	✗ No change
int	✗ No change
long	✗ No change
float	✗ No change
double	✗ No change
char	✗ No change
boolean	✗ No change
Java source code	Generally unaffected
Java bytecode	Generally unaffected
64-bit x86 JVM	✓ Supported
32-bit x86 JVM	✗ Official OpenJDK port removed
JNI/JNA	□ May require migration
Native libraries	□ Must match architecture

12. Alternate features / alternatives

Since **Remove 32-bit x86 Port** is a platform decision rather than a programming-language API, the "alternate feature" is better understood as an **alternative deployment/runtime strategy**.

Alternative	When to use	Advantages	Disadvantages
64-bit x86-64 Java 25	Modern Intel/AMD systems	Direct Java 25 migration	Requires 64-bit OS/hardware
64-bit ARM64 Java 25	ARM servers/desktops	Modern architecture, broad	Requires ARM64 infrastructure

		cloud support	
Older Java release with 32-bit support	Legacy hardware must remain	Keeps old environment operational	Eventually reaches end of support
Java 21 LTS on supported 64-bit platform	Enterprise wants LTS	Long-term support release	Doesn't solve a 32-bit hardware requirement
Java 17 LTS on supported platform	Older enterprise compatibility	Mature ecosystem	Older Java feature set
Vendor-supported legacy JDK	Specialized legacy system	May preserve old platform support	Vendor-specific lifecycle/support
Hardware/OS migration	Long-term modernization	Removes dependency on obsolete architecture	Cost and migration effort
Containerization on 64-bit infrastructure	Modern application deployment	Reproducible deployment	Cannot make a 32-bit-only host magically support a 64-bit JVM

13. Java 25 vs older Java on 32-bit x86

Requirement	Java 25	Older Java release
32-bit x86 OpenJDK port	✗	Depending on release/build, potentially ✓
x86-64	✓	✓
Modern Java features	✓	Depends on version
Modern JVM development	✓	Limited compared with Java 25
Legacy hardware	✗	Potentially ✓
Long-term modernization	✓	✗
Best for new applications	✓	✗

Best for unavoidable legacy hardware	✗	Potentially ✓
Future migration pressure	Higher now	Deferred

14. Migration checklist

Step	Action
1	Identify whether the current JVM is 32-bit or 64-bit
2	Check CPU architecture
3	Check operating-system architecture
4	Inventory JNI/JNA/native libraries
5	Obtain 64-bit versions of native dependencies
6	Upgrade the OS if necessary
7	Install a supported 64-bit Java 25 distribution
8	Run unit/integration tests
9	Test native integrations
10	Test memory-intensive workloads
11	Test production-like deployment
12	Monitor performance and memory
13	Roll out gradually

A particularly important check is:

Application
 ↓
 JVM
 ↓
 OS
 ↓
 CPU
 ↓

Native libraries

The entire stack needs compatible architecture support.

15. Overall assessment

Category	Rating	Explanation
OpenJDK maintainability	★ ★ ★ ★ ★	Removes obsolete architecture-specific maintenance
Modern infrastructure alignment	★ ★ ★ ★ ★	Matches today's predominantly 64-bit environments
Developer benefit	★ ★ ★ ☆ ☆	Mostly indirect; developers don't normally interact with the removed port
Legacy compatibility	★ ☆ ☆ ☆ ☆	Negative for 32-bit x86 systems
Modern server benefit	★ ★ ★ ★ ★	Little/no downside for 64-bit deployments
Migration simplicity	★ ★ ★ ☆ ☆	Easy for pure-Java applications; harder with native dependencies
Embedded-system suitability	★ ★ ☆ ☆ ☆	Can be problematic for old 32-bit devices
Long-term maintainability	★ ★ ★ ★ ★	Encourages migration away from obsolete hardware
Application source-code impact	★ ★ ★ ★ ★	Usually minimal
Infrastructure impact	★ ★ ☆ ☆ ☆	Potentially significant for legacy systems

Final decision table

Your environment	Recommendation
64-bit Intel/AMD server	☐ Use Java 25

64-bit desktop/laptop	☐ Use Java 25
x86-64 cloud VM	☐ Use Java 25
ARM64 server	☐ Use appropriate Java 25 build
32-bit x86 server	☐ Do not use Java 25 OpenJDK
32-bit legacy desktop	☐ Do not use Java 25 OpenJDK
32-bit industrial machine	☐ Do not upgrade without hardware/platform migration
32-bit JNI application	☐ Migrate native libraries first
New application	☐ Target a modern 64-bit Java runtime
Legacy application with no upgrade requirement	☐ Evaluate whether migration is worth the cost
Enterprise modernization	☐ Good opportunity to migrate to 64-bit Java 25

Bottom line

JEP 503 is primarily a cleanup and modernization change. It does not improve application performance simply by being present. Its practical message is:

Java 25 targets modern 64-bit platforms; 32-bit x86 is now legacy infrastructure.

For a **64-bit x86 Java application**, you generally don't need to change anything because of this JEP. For a **32-bit x86 deployment**, however, Java 25 requires a platform migration rather than a simple JVM upgrade.

Java 25 — Structured Concurrency (Fifth Preview)

UniqueClassName: StructuredConcurrency

JEP: 505

Java Version: 25

Year: 2025

Status: Fifth Preview

Structured Concurrency provides a structured way to manage **concurrent tasks that belong to the same overall operation**. It treats related concurrent work as a single unit, making **task lifetime, cancellation, failure handling, and result collection** easier to reason about.

It works particularly well with **virtual threads** introduced in Java 21.

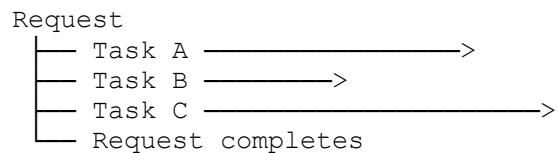
1. Feature overview

Aspect	Details
Feature	Structured Concurrency
UniqueClassName	StructuredConcurrency
JEP	505
Java Version	25
Year	2025
Status	Fifth Preview
Main API	StructuredTaskScope
Package	java.util.concurrent
Primary purpose	Manage related concurrent tasks as one structured operation
Typical tasks	Parallel HTTP calls, database queries, service calls, file operations
Main concepts	Fork, join, cancellation, failure propagation
Typical execution model	Parent task creates child tasks, waits for them, then completes
Works well with	Virtual threads
Main benefit	Makes concurrent code easier to understand, cancel, debug, and maintain

Main limitation	Still a preview API in Java 25

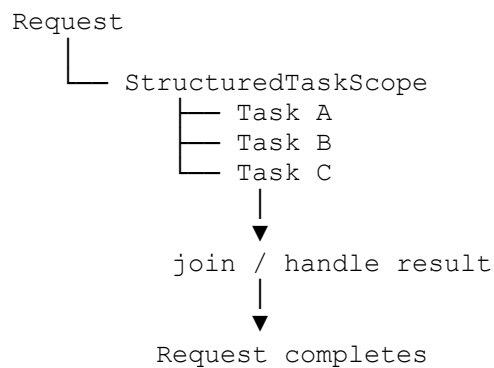
2. Core idea

Traditional concurrency can easily become unstructured:



Some tasks may continue running after the original request has already failed.

Structured concurrency tries to enforce:



The parent operation owns the child tasks.

This creates a useful rule:

A concurrent task should not outlive the operation that created it.

3. Advantages

Advantage	Explanation	Real-world benefit
Simpler concurrency model	Related tasks are grouped into one scope	Easier to understand application flow
Task lifetime	Child tasks belong to their	Prevents orphaned background

management	parent scope	work
Automatic cancellation	Scope policies can cancel remaining tasks	Saves resources after failure
Failure propagation	Task failures can be handled as part of the overall operation	Avoids silently ignored failures
Parallel execution	Independent tasks can execute concurrently	Lower overall latency
Works with virtual threads	Natural companion to lightweight virtual threads	Large numbers of concurrent I/O operations become easier to manage
Better readability	Concurrent operations can resemble sequential code	Easier maintenance
Clear ownership	Parent operation owns its child tasks	Better lifecycle management
Improved error handling	Scope can coordinate success/failure	Fewer race-condition-prone error paths
Cancellation propagation	Parent cancellation can propagate to children	Prevents unnecessary work
Resource efficiency	Unneeded tasks can be stopped	Lower CPU/network/database usage
Observability	Structured task hierarchy provides a clearer execution model	Easier debugging and tracing
Avoids thread-management boilerplate	Less direct executor/future coordination	Cleaner application code
Composability	Nested scopes can represent nested operations	Useful in complex service architectures

4. Disadvantages

Disadvantage	Explanation	Impact
Preview API	Java 25 still treats it as a	API/design may change

	preview	
Requires learning	Developers must understand structured concurrency concepts	Training required
Not suitable for every task	Some background work intentionally outlives its parent	Another concurrency model may be better
Migration effort	Existing <code>ExecutorService/CompletableFuture</code> code may require redesign	Not always a drop-in replacement
Cancellation semantics	Cancellation can be surprising if developers don't understand scope policies	Incorrect behavior is possible
Blocking operations still matter	Virtual threads make blocking cheaper but don't eliminate slow external systems	Database/API limits remain
Scope lifetime matters	Child tasks must fit naturally within the scope	Poorly designed scopes can become awkward
Preview compatibility	Requires preview compilation/runtime configuration	Operational complexity
Debugging can still be complex	Concurrent failures remain inherently more difficult than sequential failures	Requires good observability
Not a performance guarantee	Concurrency can increase load on downstream systems	Parallel execution isn't always faster
Over-parallelization	Too many concurrent operations can overload databases/APIs	Must respect resource limits
Not a replacement for every executor	Long-running independent jobs don't naturally fit structured scopes	Continue using appropriate job/executor mechanisms

5. When to use Structured Concurrency

Situation	Use?	Why
Multiple independent API calls	✓ Yes	Calls can execute concurrently
Microservice aggregation	✓ Yes	Multiple services can be queried together
Parallel database queries	✓ Yes	Independent queries can run concurrently
Fan-out/fan-in processing	✓ Yes	Natural fit for structured task scopes
Request-scoped concurrent work	✓ Yes	Child work should finish/cancel with request
Parallel file processing	✓ Yes	Independent files can be processed concurrently
Calling several external services	✓ Yes	Reduces overall latency
Race among equivalent services	✓ Yes	First successful result may be sufficient
Concurrent validation	✓ Yes	Multiple independent validations can run together
Batch operation with common deadline	✓ Yes	Scope can coordinate completion/cancellation
Dynamic task tree	✓ Yes	Nested structured operations fit naturally
Background job intended to outlive request	✗ No	Lifetime intentionally exceeds parent
Scheduled recurring jobs	✗ Usually	Scheduler is more appropriate
Message consumers running indefinitely	✗ Usually	Independent lifecycle
Application-wide daemon	✗ No	It shouldn't belong to one request

Queue workers	✗ Usually	Long-lived worker model is more appropriate
Simple sequential code	✗ No	Concurrency adds unnecessary complexity
Shared mutable-state-heavy code	☐ Carefully	Concurrency may increase synchronization complexity

6. Basic example

Consider a customer dashboard that needs:

- customer profile,
- orders,
- recommendations.

Sequential approach:

```

Profile API      200 ms
Orders API       300 ms
Recommendation   250 ms
-----
Total            ~750 ms

```

With concurrent execution:

```

Profile API      ——— 200 ms
Orders API       ——— 300 ms
Recommendation   ——— 250 ms
      |
      v
    ~300 ms

```

The tasks are independent, so executing them concurrently can reduce latency.

7. Conceptual Java example

With the Java 25 preview API, the basic structure is approximately:

```
try (var scope = StructuredTaskScope.open()) {
```

```

var profile =
    scope.fork(() -> getCustomerProfile());

var orders =
    scope.fork(() -> getOrders());

var recommendations =
    scope.fork(() -> getRecommendations());

scope.join();

var profileResult = profile.get();
var orderResult = orders.get();
var recommendationResult = recommendations.get();

return new Dashboard(
    profileResult,
    orderResult,
    recommendationResult
);
}

```

The important structure is:

```

open scope
↓
fork task A
fork task B
fork task C
↓
join
↓
collect results
↓
scope closes

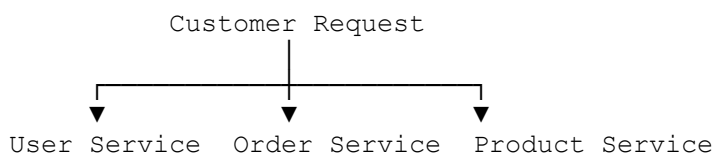
```

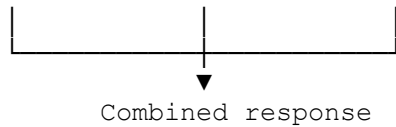
The exact API surface is preview and should be checked against the Java 25 API/JEP when implementing it.

8. Real-time Use Case 1 — Microservice aggregation

Imagine an e-commerce application.

A request needs:





Without structured concurrency

Developers may create several futures/executor tasks and manually coordinate:

Future A
Future B
Future C
↓
wait
↓
check errors
↓
cancel others
↓
combine results

With structured concurrency

Scope
├── User
├── Orders
└── Products
↓
join
↓
aggregate

Why it is useful: all three tasks belong to the same request.

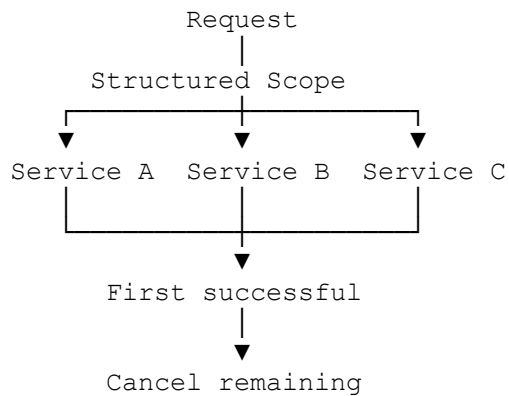
9. Real-time Use Case 2 — First successful response

Suppose an application has three equivalent services:

Service A
Service B
Service C

You need only **one successful result**.

Conceptually:



This is particularly useful for:

- replicated services,
- redundant data providers,
- multi-region APIs,
- fallback providers.

Structured concurrency provides scope policies designed around cases such as **shutdown on success** and **shutdown on failure**.

10. Real-time Use Case 3 — Parallel database queries

Suppose a reporting API needs:

Sales data
Customer data
Inventory data

If queries are independent:

Sequential:

Sales	_____		
Customer		_____	
Inventory			_____

With structured concurrency:

Sales	_____
Customer	_____
Inventory	_____

The overall operation completes when the required tasks finish.

Important warning

Concurrency does **not** mean:

"Send unlimited queries to the database."

You still need:

- database connection limits,
- query limits,
- transaction considerations,
- back-pressure,
- appropriate concurrency levels.

11. Real-time Use Case 4 — HTTP aggregation

A travel application might need:

Flight prices

Hotel prices

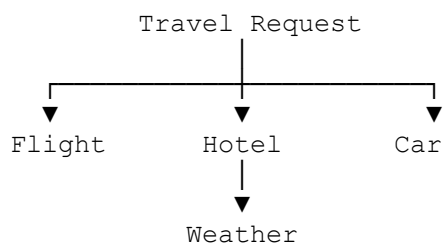
Car rental prices

Weather

Instead of:

Flight → Hotel → Car → Weather

it can execute:



Independent operations execute concurrently.

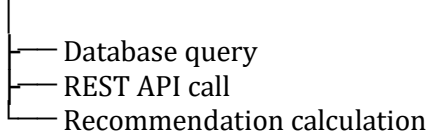
If the user cancels the request, the application can cancel the structured child operations rather than allowing unnecessary work to continue.

12. Real-time Use Case 5 — Request cancellation

This is one of the most important benefits.

Suppose:

HTTP Request



The user closes the browser.

Without good lifecycle management:

Request cancelled



Child tasks continue



Database/API resources still consumed

With structured concurrency:

Request cancelled



Scope cancelled



Child tasks cancelled



Resources released

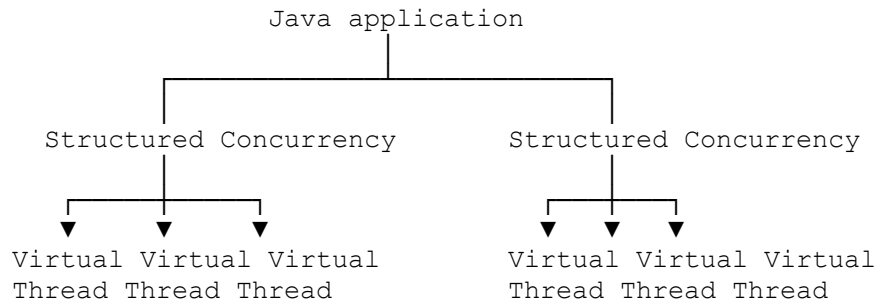
This is especially valuable for high-traffic applications.

13. Structured Concurrency + Virtual Threads

These two Java features complement each other.

Feature	Primary purpose
Virtual Threads	Make large numbers of concurrent tasks inexpensive
Structured Concurrency	Organize and manage those concurrent tasks
Combination	Lightweight concurrency + structured lifecycle management

Conceptually:



Virtual threads answer:

"How can I run many concurrent tasks efficiently?"

Structured concurrency answers:

"How should those tasks be organized and managed?"

14. Structured Concurrency vs CompletableFuture

Aspect	Structured Concurrency	CompletableFuture
Primary abstraction	Task scope	Future/pipeline
Task hierarchy	Explicit	Less explicit
Parent-child relationship	Strong	Not inherently structured
Cancellation propagation	Scope-oriented	Requires careful composition
Failure handling	Scope policies	Completion-stage mechanisms
Readability	Often simpler for request-scoped concurrency	Can become complex with large chains
Async composition	Good	Excellent
Functional pipelines	Limited	Excellent
Parallel request fan-out	Excellent	Good
Long-running independent async work	Less suitable	Often better

Preview in Java 25	Yes	No
Best use	Related tasks with common lifecycle	Complex asynchronous pipelines

15. Structured Concurrency vs ExecutorService

Aspect	Structured Concurrency	ExecutorService
Task lifecycle	Scope-based	Executor-based
Ownership	Parent scope	Executor
Cancellation	Structured	Manually coordinated
Task hierarchy	Explicit	Usually implicit
Thread pooling	Not its main purpose	Core purpose
Virtual-thread integration	Excellent	Can use virtual-thread executors
Request-scoped work	Excellent	Requires additional management
Long-running jobs	Poor fit	Excellent
Scheduling	Not intended	Use scheduled executors for scheduling
Simplicity	High for structured tasks	Familiar and flexible
Legacy compatibility	Newer model	Mature

16. Structured Concurrency vs parallel streams

Aspect	Structured Concurrency	Parallel Streams
Main purpose	Concurrent tasks	Parallel data processing
HTTP calls	Excellent	Poor fit
Database calls	Good	Usually inappropriate

CPU-intensive collection processing	Possible	Excellent
Task cancellation	Stronger model	Limited
Task lifecycle	Explicit	Stream-oriented
Error handling	Structured	Stream exception model
Virtual threads	Works naturally	Not the primary model
Best use	I/O-oriented concurrent operations	CPU/data parallelism

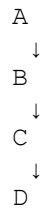
17. Alternate features

Requirement	Recommended alternative
Independent request-scoped concurrent tasks	Structured Concurrency
Complex async pipeline	CompletableFuture
General task execution	ExecutorService
Lightweight concurrent I/O	Virtual Threads
CPU-intensive collection processing	Parallel Streams / ForkJoinPool
Long-running background jobs	Executor / job queue
Scheduled tasks	ScheduledExecutorService
Distributed asynchronous work	Messaging system / job queue
Reactive streams with backpressure	Reactive Streams / Project Reactor / similar framework
Simple one-off task	Virtual thread or executor
Dynamic cache operations	Concurrent collections/cache

18. When CompletableFuture may be better

Structured concurrency is not a universal replacement.

Use `CompletableFuture` when you need something like:

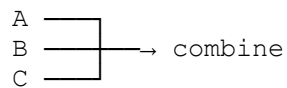


with complex asynchronous transformations:

```
future
  .thenApply(...)
  .thenCompose(...)
  .thenCombine(...)
  .exceptionally(...)
```

It is particularly useful when the **asynchronous pipeline itself is the abstraction**.

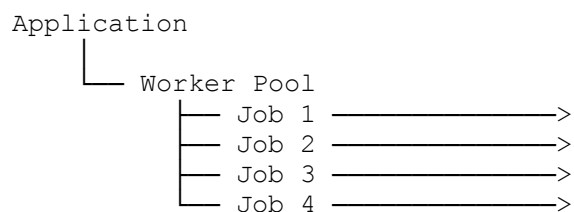
Structured concurrency is generally more natural for:



where the tasks are siblings belonging to one operation.

19. When `ExecutorService` may be better

Use an executor when tasks intentionally have an independent lifecycle:



Examples:

- background jobs,
- queue consumers,
- scheduled processing,

- long-running workers,
- application-wide task pools.

These tasks aren't necessarily children of a single request, so structured concurrency isn't the natural abstraction.

20. Important architectural rule

A useful decision rule is:

Question	If YES
Does the task belong to the current request/operation?	Consider Structured Concurrency
Should the task finish before its parent operation finishes?	Consider Structured Concurrency
Should cancellation propagate to child tasks?	Consider Structured Concurrency
Are tasks siblings contributing to one result?	Consider Structured Concurrency
Should tasks continue after the request finishes?	Consider an independent executor/job system
Is the task a permanent worker?	Use worker/executor architecture
Is the work scheduled periodically?	Use scheduler
Is it a complex asynchronous pipeline?	Consider CompletableFuture
Is it CPU-bound data parallelism?	Consider parallel streams/ForkJoinPool

21. Performance considerations

Scenario	Expected effect
Independent network calls	☐ Often significantly lower latency
Independent database calls	☐ Potential latency reduction, but DB capacity is limiting

CPU-bound tasks	☐ Depends on CPU/core availability
Many blocking I/O operations	☐ Excellent candidate with virtual threads
First-use initialization	☐ Depends on application
Downstream service has strict rate limit	☐ Parallelism can hurt
Very small tasks	☐ Concurrency overhead may outweigh benefits
Thousands of independent I/O operations	☐ Potentially excellent, with appropriate limits

22. Common mistakes

Mistake	Problem	Better approach
Forking unlimited tasks	Can overwhelm downstream systems	Apply concurrency limits
Using it for permanent workers	Wrong lifecycle model	Executor/job system
Assuming concurrency always improves performance	Resource contention can make it slower	Benchmark
Ignoring cancellation	Resources may remain active	Design cancellation propagation
Sharing mutable state everywhere	Race conditions	Prefer immutable results
Making every method concurrent	Adds unnecessary complexity	Use concurrency only where useful
Treating preview API as permanently stable	API may change	Isolate preview usage
Ignoring downstream timeouts	Requests can remain blocked	Use deadlines/timeouts
Creating huge task trees without reason	Difficult to understand	Keep scopes aligned with business operations

23. Production decision matrix

Application requirement	Structured Concurrency
Microservice aggregation	☐ Excellent
Parallel REST calls	☐ Excellent
Parallel DB queries	☐ Very good
mTLS/network operations	☐ Good
File/I/O processing	☐ Good
Request cancellation	☐ Excellent
First-success race	☐ Excellent
Fan-out/fan-in	☐ Excellent
Virtual-thread application	☐ Excellent companion
Complex async transformation pipeline	☐ CompletableFuture may be better
Scheduled jobs	☐ Use scheduler
Long-running workers	☐ Use executor/job system
Message queue consumers	☐ Usually use worker model
CPU-heavy parallel computation	☐ Consider ForkJoinPool/parallelism
Simple sequential application	☐ Unnecessary
Java 8/11/17 application	☐ Not available
Stable public library API	☐ Preview status requires caution

24. Overall assessment

Category	Rating	Explanation
Readability	★★★★★	Makes concurrent workflows easier to understand

Lifecycle management	★★★★★	Strong parent-child task relationship
Cancellation	★★★★★	Major strength
Error handling	★★★★★	Failures can be handled at scope level
Microservice use	★★★★★	Excellent for service aggregation
Virtual-thread integration	★★★★★	Natural companion
Performance potential	★★★★☆	Particularly strong for I/O-bound workloads
Flexibility	★★★★☆	Less suitable for intentionally independent tasks
Learning curve	★★★☆☆	Requires new concurrency concepts
API maturity	★★★☆☆	Still preview in Java 25
Legacy compatibility	★★☆☆☆	Requires newer Java
Background-job suitability	★★☆☆☆	Independent job systems are generally better

Final recommendation

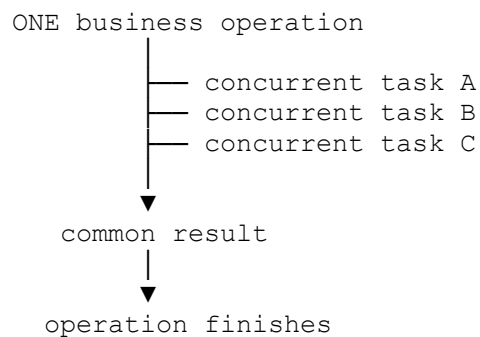
Scenario	Recommendation
REST API calls that belong to one request	🔗 Use Structured Concurrency
Microservice aggregation	🔗 Use Structured Concurrency
Parallel DB/API calls	🔗 Use Structured Concurrency
Request cancellation should cancel child work	🔗 Use Structured Concurrency
First successful response wins	🔗 Use Structured Concurrency
Virtual-thread-based application	🔗 Strong combination
Complex asynchronous pipelines	🔗 Consider CompletableFuture

Long-running background jobs	☑/☑ Use executor or job queue
Scheduled processing	☑ Use a scheduler
Permanent worker threads	☑ Use an executor/worker architecture
Simple sequential operation	☑ Don't introduce concurrency unnecessarily
Java 17 or earlier	☑ Not available
Production application requiring stable API today	☑ Preview status requires caution

Bottom line

Structured Concurrency is best understood as a lifecycle-management model for concurrent work, not simply another way to create threads.

Its strongest use case is:



If A, B, and C belong to the same operation, Structured Concurrency is an excellent candidate.

If A, B, and C must continue independently after their parent operation finishes, use an executor, scheduler, queue, or another independent background-processing mechanism instead.

Java 25 — Scoped Values

UniqueClassName: ScopedValues

JEP: 506

Java Version: 25

Year: 2025

Status: Final

Scoped Values provide a way to make **immutable data available to methods deep in a call chain without explicitly passing that data through every method parameter**.

They are particularly useful for **context propagation** in applications using **virtual threads and structured concurrency**.

Typical examples include:

- request context
- authenticated user/context
- transaction or correlation information
- security context
- logging/trace information
- locale
- configuration associated with an operation

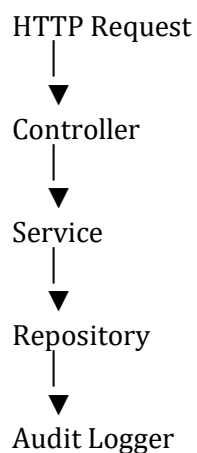
1. Feature overview

Aspect	Details
Feature	Scoped Values
UniqueClassName	ScopedValues
JEP	506
Java Version	25
Year	2025
Status	Final
Main API	ScopedValue<T>
Package	java.lang
Primary purpose	Share immutable context through a bounded call scope
Alternative to	Some uses of ThreadLocal

Main characteristic	Immutable, scoped, inheritable context
Mutation	✗ Value cannot be reassigned within the scope
Lifetime	Limited to a lexical/dynamic scope
Thread support	Designed particularly well for virtual threads
Structured concurrency integration	Excellent
Typical use	Request context and operation-specific metadata

2. The problem Scoped Values solve

Suppose a request has:



You need the same requestId everywhere.

Without contextual storage:

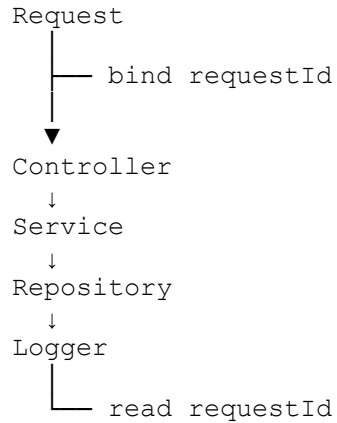
```

controller(requestId)
    → service(requestId)
        → repository(requestId)
            → logger(requestId);
  
```

The requestId gets passed through methods that don't actually need to understand it.

This is sometimes called **parameter pollution**.

Scoped Values allow:



The intermediate methods don't need to accept requestId merely to pass it onward.

3. Basic example

A simplified example:

```
static final ScopedValue<String> REQUEST_ID =  
    ScopedValue.newInstance();
```

Bind a value for a particular operation:

```
ScopedValue.where(REQUEST_ID, "REQ-12345")  
    .run(() -> processRequest());
```

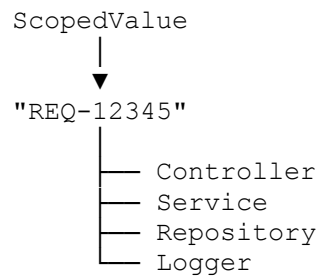
Any method executed within that scope can access it:

```
static void processRequest() {  
    service();  
}  
  
static void service() {  
    repository();  
}  
  
static void repository() {  
    System.out.println(REQUEST_ID.get());  
}
```

Output:

REQ-12345

Conceptually:



The value is **bound for the scope**, rather than stored as mutable thread-local state.

4. Advantages

Advantage	Explanation	Real-world benefit
Avoids parameter pollution	Context doesn't need to be passed through every method	Cleaner APIs
Immutable	Once bound, the value cannot be changed by code inside the scope	Safer context propagation
Bounded lifetime	Value exists only within its scope	Prevents accidental long-term retention
Excellent for request context	Request metadata can be available throughout the call tree	Cleaner web-service architecture
Works with virtual threads	Designed with modern Java concurrency in mind	Good fit for high-concurrency applications
Works with structured concurrency	Context can naturally accompany structured tasks	Easier concurrent request processing
Inheritance	Child execution can receive the parent's scoped context according to the API's inheritance semantics	Convenient context propagation

No explicit cleanup pattern like ThreadLocal	Scope controls availability	Less lifecycle-management code
Strong encapsulation	Methods can access only explicitly defined scoped context	Better design than global mutable state
Concurrency safety	Immutable context avoids many mutable-state races	Fewer synchronization concerns
Readability	Business parameters remain business parameters	Cleaner method signatures
Debugging	Context can be consistently available across call hierarchy	Easier correlation/tracing
Good observability support	Correlation IDs and trace context can flow through operations	Better logs/traces
Lexical structure	Binding is visible at the point where the scope is created	Easier reasoning about lifetime
Final API in Java 25	No longer a preview feature	Suitable for standard Java 25 development

5. Disadvantages

Disadvantage	Explanation	Impact
Can hide dependencies	A method may depend on a Scoped Value without showing it in parameters	API dependencies become less explicit
Not suitable for mutable state	Scoped Values are intended for stable contextual information	Use another mechanism for mutable shared state
Requires discipline	Developers can misuse contextual values as global variables	Architecture can become difficult to understand
Debugging hidden	A method signature doesn't	More effort may be required to

context	reveal all context it depends on	understand call paths
Scope limitations	Value is only available within its scope	Must design boundaries correctly
Not a database/cache replacement	Doesn't manage persistent or dynamic application data	Need proper storage mechanisms
Poor fit for request-independent state	Global configuration shouldn't necessarily be represented as scoped context	Use configuration/immutable fields
Can encourage overuse	Developers might put ordinary method arguments into Scoped Values	Reduces clarity
Context naming matters	Poorly named scoped variables make code confusing	Establish conventions
Migration considerations	Replacing all ThreadLocal usage isn't always appropriate	Evaluate each use case individually

6. Scoped Values vs ThreadLocal

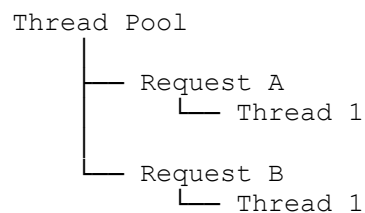
This is the most important comparison.

Aspect	ThreadLocal	ScopedValue
Introduced	Java 1.2 era	Java 25 final
Mutable by owner	✓ Yes	✗ No
Lifetime	Associated with thread state	Associated with scope
Explicit bounded scope	✗ Not inherent	✓
remove() often needed	✓ Yes	✗ Scope handles lifetime

Risk of stale context	Higher with thread pools if mismanaged	Lower due to scoped lifetime
Context inheritance	Traditional InheritableThreadLocal has inheritance semantics	Designed for scoped inheritance
Virtual threads	Works, but not the primary design goal	Designed for modern concurrency
Structured concurrency	Can require additional context handling	Natural fit
Mutation	Possible	Not allowed
Best use	Thread-associated mutable state	Immutable operation context
Memory/lifecycle management	Developer responsibility	Scope-oriented
Accidental leakage	Possible	Much harder when used correctly
Recommendation for new context propagation	Often not first choice	Prefer Scoped Values where semantics fit

7. Why Scoped Values are safer than ThreadLocal for context

Consider a thread pool:



With a poorly managed ThreadLocal:

```

Request A
  ↓
ThreadLocal = UserA
  ↓
Request completes

```

```

↓
Thread reused
↓
Request B
↓
Potential stale ThreadLocal state

```

This is why `ThreadLocal.remove()` is important in traditional thread-pool designs.

Scoped Values instead provide:

```

Request A
↓
ScopedValue = UserA
↓
scope ends
↓
binding ends
↓
Request B
↓
new scope

```

The **scope defines the lifetime** of the context.

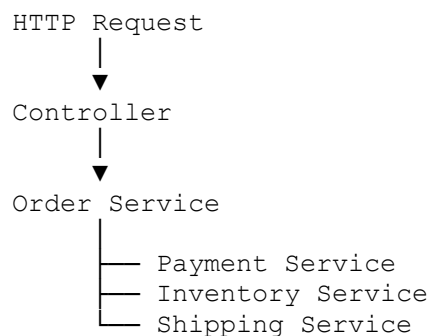
8. When to use Scoped Values

Situation	Use?	Reason
Request ID	☑ Yes	Common immutable request context
Trace/correlation ID	☑ Yes	Needed deep in call stack
Authenticated user context	☑ Yes	Contextual and operation-scoped
Security context	☑ Yes	Useful when immutable for the operation
Locale	☑ Yes	Contextual request information
Tenant ID	☑ Yes	Multi-tenant request context
Logging context	☑ Yes	Correlates logs
Transaction context	☑ Potentially	Good if the transaction context is immutable and scope-bound

Request deadline	☒ Yes	Useful across nested operations
Feature/configuration context	☒ Sometimes	Good when fixed for the operation
Virtual-thread applications	☒ Yes	Strong use case
Structured concurrency	☒ Excellent	Natural context propagation
Mutable application state	☒ No	Use appropriate mutable/concurrent storage
Cache	☒ No	Use cache
Database entity storage	☒ No	Use repository/database
Global configuration	☒ Usually no	Prefer immutable configuration object
Request-specific mutable state	☒	Use explicit request context object or suitable concurrent structure

9. Real-time Use Case 1 — Request ID

A production REST service may have:



Every log needs:

`requestId = REQ-981273`

Instead of:

`processOrder(requestId)`

```
validateOrder(requestId)
reserveInventory(requestId)
chargePayment(requestId)
shipOrder(requestId)
```

you can establish request context at the boundary:

```
ScopedValue.where(REQUEST_ID, "REQ-981273")
    .run(() -> processOrder());
```

Then deep code can retrieve the ID.

Benefit

Business method signatures remain:

```
processOrder()
chargePayment()
reserveInventory()
```

rather than becoming:

```
processOrder(requestId)
chargePayment(requestId)
reserveInventory(requestId)
```

10. Real-time Use Case 2 — Multi-tenant application

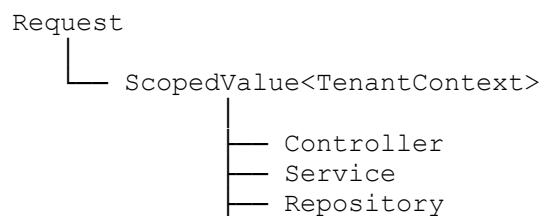
Suppose a SaaS application handles:

Tenant A
Tenant B
Tenant C

A request is associated with:

tenantId = TENANT-42

The application can establish:



└─ Audit

The tenant context remains available throughout the operation.

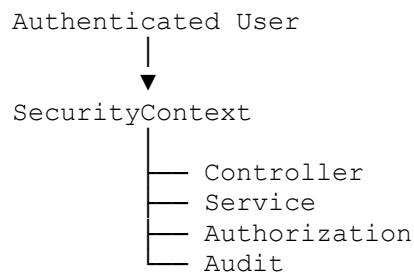
This can be useful for:

- tenant-aware database access,
- authorization,
- audit logging,
- feature configuration,
- metrics.

Important: Scoped Values do not themselves enforce tenant isolation. Your application must still enforce authorization and data-access boundaries correctly.

11. Real-time Use Case 3 — Security context

Consider:



A scoped immutable security context could contain information such as:

User ID
Roles
Tenant
Authentication information

Deep service methods can inspect the context without every method receiving a `SecurityContext` parameter.

This is especially useful when many layers need the same request-specific security information.

12. Real-time Use Case 4 — Distributed tracing

A request enters:

```
API Gateway
  ↓
Order Service
  ↓
Payment Service
  ↓
Inventory Service
```

A tracing context might contain:

```
traceId = abc123
spanId  = xyz789
```

Scoped Values can provide contextual access:

```
Request
├── TraceContext
│   ├── Controller
│   ├── Service
│   ├── Database
│   └── HTTP client
```

Every component can obtain the current trace context without passing it through every method.

This is a particularly natural use because tracing context is:

- contextual,
- immutable for a particular scope,
- needed deep in the call tree.

13. Real-time Use Case 5 — Request deadline

Suppose an HTTP request has a deadline:

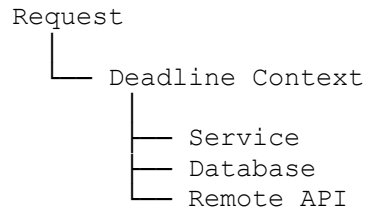
Deadline = 2 seconds

Several downstream operations need to know the remaining time.

Instead of:

```
service(deadline)
repository(deadline)
remoteCall(deadline)
```

a scoped deadline can be established:

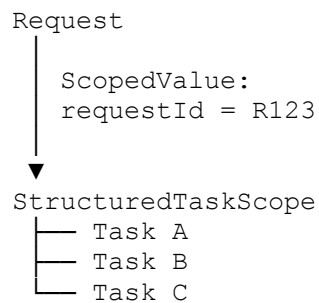


This is particularly useful in combination with structured concurrency.

14. Scoped Values + Structured Concurrency

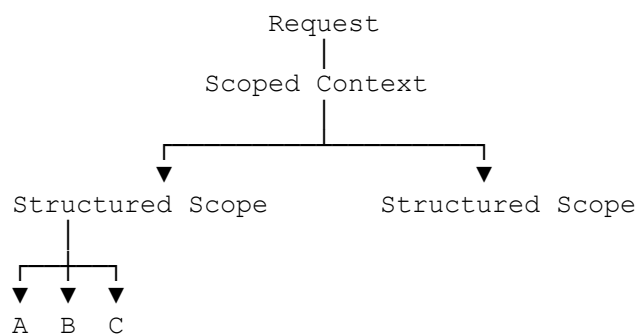
This is one of the strongest Java 25 combinations.

Suppose:



The child tasks can operate with the relevant scoped context.

Conceptually:



This combination gives:

Scoped Values → context propagation

Structured Concurrency → task lifecycle

Virtual Threads → inexpensive concurrent execution

Together they form a powerful model for modern server applications.

15. When NOT to use Scoped Values

Requirement	Why not	Better alternative
Counter that changes frequently	Scoped Values aren't mutable shared state	AtomicInteger, LongAdder, etc.
Mutable shopping cart	State needs modification	Explicit object/request state
Application cache	Needs multiple values/eviction	Cache implementation
Database data	Persistent external state	Database/repository
Global configuration	Doesn't need per-scope semantics	Immutable configuration object
Per-user persistent preferences	Scope is too short-lived	Database/configuration store
Cross-request state	Scoped Values are operation-scoped	Appropriate shared storage
Event queue	Needs independent lifecycle	Queue/message broker
Background worker state	Worker outlives request	Executor/worker architecture
Object-specific state	Better represented by fields	final/instance fields
Ordinary method input	Hiding it makes API less explicit	Method parameter

16. A major design warning

Do **not** turn every parameter into a Scoped Value.

Bad design:

ScopedValue<Customer>
ScopedValue<Product>
ScopedValue<Amount>
ScopedValue<Address>
ScopedValue<Order>
ScopedValue<Payment>

This can make the application look like it has a collection of hidden global variables.

Prefer normal parameters for **business inputs**:

calculateTax(order, customer, amount);

Use Scoped Values for **context**:

requestId
tenantId
traceId
security context
deadline
locale

A useful rule is:

If the value describes the environment in which an operation runs, Scoped Value may be appropriate. If it is the actual data being processed, prefer a method parameter.

17. Scoped Value vs method parameter

Data	Preferred approach
orderId being processed	Method parameter
amount being calculated	Method parameter
customer being created	Method parameter
requestId	Scoped Value
traceId	Scoped Value
tenantId	Often Scoped Value
authenticatedUser	Often Scoped Value
deadline	Scoped Value

locale	Potentially Scoped Value
databaseConnection	Usually explicit resource/context management
cache	Application component
configuration	Immutable configuration object
transaction context	Context mechanism appropriate to transaction framework

18. Scoped Values vs common alternatives

Alternative	Best use	Scoped Values better when	Alternative better when
Method parameters	Business data	Context needs to cross many layers	Data is directly required by method
ThreadLocal	Thread-associated state	Immutable bounded context	Mutable thread-specific state is genuinely required
InheritableThreadLocal	Legacy child-thread context	Modern structured/virtual-thread context	Legacy code requires thread-local inheritance
Instance fields	Object state	Context spans many unrelated objects	State belongs naturally to an object
Global/static variable	Very rare constants	Context is request/operation-specific	Value truly is global and immutable
CompletableFuture context	Async pipelines	Context follows structured execution	Complex async pipeline is the primary abstraction
Framework context	Framework-managed request state	You want standard Java context mechanism	Framework already has a well-designed context propagation system
Explicit Context object	Many related context fields	You need controlled contextual access	Context contains substantial business data

19. Scoped Values vs ThreadLocal: practical decision

Question	ThreadLocal	Scoped Value
Is the value immutable?	Optional	Yes, intended
Is it request-scoped?	Possible	Excellent
Does the value need to be changed?	✓	✗
Should it automatically disappear after scope?	Requires discipline	Yes
Is the application heavily virtual-thread based?	Possible	Strong candidate
Is structured concurrency used?	Possible	Excellent combination
Is it legacy framework integration?	Often	Depends
Is the value truly thread-local rather than operation-scoped?	Yes	Maybe not
Does the value need set()/remove() semantics?	Yes	No

20. Performance considerations

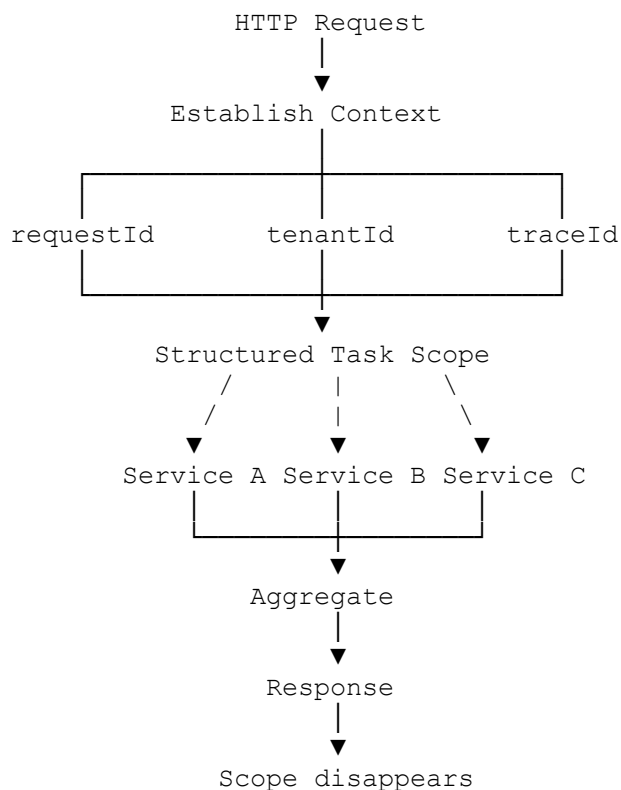
Area	Scoped Values
Lookup overhead	Designed to be efficient
Memory lifetime	Bounded by scope
Thread-pool leakage risk	Lower than manually managed ThreadLocal state
Virtual-thread compatibility	Excellent

Context copying	Designed to support scoped inheritance
Mutable synchronization	Avoided for the value itself
Performance guarantee	Should still be benchmarked in real workloads
Main performance benefit	More efficient/cleaner context propagation model rather than simply "faster than everything"

Don't adopt Scoped Values solely because of a presumed microbenchmark advantage. Their biggest benefit is **correctness, lifecycle, and concurrency design**.

21. Architecture example

For a modern Java 25 microservice:



This is one of the clearest architectural use cases for Scoped Values.

22. Real-time enterprise examples

Application	Scoped Value	Why
Banking	transactionContext	Propagate transaction-related context
E-commerce	tenantId	Multi-tenant processing
Payment system	requestId	Correlate payment operations
Microservices	traceId	Distributed tracing
Healthcare platform	securityContext	Propagate authorization context
SaaS	tenantContext	Tenant-aware operations
Logistics	requestDeadline	Propagate operation deadlines
Retail	locale	Request-specific formatting/localization
Audit platform	auditContext	Consistent operation metadata
API gateway	correlationId	End-to-end request correlation
Messaging system	messageContext	Context associated with processing a message

23. Best-practice rules

Rule	Recommendation
Use for contextual information	✓
Prefer immutable values	✓
Keep scope as narrow as practical	✓
Use descriptive names	✓
Document important scoped dependencies	✓
Combine with structured concurrency	✓

Use with virtual threads where appropriate	✓
Pass business data explicitly	✓
Use it as a global mutable variable	✗
Store large mutable object graphs	✗
Replace every method parameter	✗
Use it as a cache	✗
Use it for permanent application state	✗
Assume it replaces every ThreadLocal	✗

24. Overall assessment

Category	Rating	Explanation
Context propagation	★★★★★	Excellent
Concurrency safety	★★★★★	Immutable scoped model
Virtual-thread integration	★★★★★	Designed for modern Java concurrency
Structured concurrency integration	★★★★★	Excellent
Request context	★★★★★	One of its strongest use cases
Readability	★★★★☆	Cleaner signatures, but dependencies become less explicit
Lifecycle management	★★★★★	Scope naturally limits lifetime
Performance potential	★★★★☆	Efficient contextual access, but benchmark critical workloads
Flexibility	★★★☆☆	Intentionally not mutable
Migration from ThreadLocal	★★★★☆	Strong candidate for many context use cases

Suitability for business data	☆☆☆☆☆	Method parameters are usually clearer
API maturity	☆☆☆☆	Final in Java 25

25. Final decision table

Scenario	Recommendation
Request ID	🔍 Use Scoped Values
Correlation ID	🔍 Use Scoped Values
Distributed tracing context	🔍 Use Scoped Values
Tenant context	🔍 Use Scoped Values
Authenticated/security context	🔍 Use Scoped Values
Request deadline	🔍 Use Scoped Values
Locale/request context	🔍 Consider Scoped Values
Virtual-thread application	🔍 Strong candidate
Structured Concurrency application	🔍 Excellent combination
Mutable counter	🔍 Use AtomicInteger/LongAdder etc.
Mutable request state	🔍 Prefer explicit context/object
Business method argument	🔍 Use normal parameter
Application cache	🔍 Use a cache
Persistent state	🔍 Use database/storage
Long-running background job context	🔍 Use an appropriate job/executor context
Legacy ThreadLocal with mutation requirements	🔍 Keep ThreadLocal if semantics genuinely require it

Bottom line

Scoped Values are best for immutable contextual information that should be available throughout a bounded operation.

The three Java 25 concepts fit together particularly well:

Java 25 Application	
Scoped Values	→ "What context am I in?"
Structured Concurrency	→ "What tasks belong to this operation?"
Virtual Threads	→ "How can I run many concurrent tasks cheaply?"

So, for a modern Java 25 server application, a very strong pattern is:

Scoped Values for request/operation context + Structured Concurrency for task lifecycle + Virtual Threads for scalable blocking I/O.

Java 25 — Primitive Types in Patterns (Third Preview)

UniqueClassName: PrimitiveTypesPatterns
JEP: 488
Java Version: 25
Year: 2025
Status: Third Preview

This feature extends Java's **pattern matching** so that primitive values such as int, long, double, etc. can participate directly in instanceof and switch patterns, including **type patterns and primitive type patterns**.

The important goal is to make pattern matching more expressive while reducing unnecessary boxing/unboxing and allowing primitive values to be handled naturally alongside reference types.

1. Feature overview

Aspect	Details
Feature	Primitive Types in Patterns

UniqueClassName	PrimitiveTypesPatterns
JEP	488
Java Version	25
Year	2025
Status	Third Preview
Main language feature	Pattern matching involving primitive types
Main constructs	switch, case, primitive type patterns, guarded/conditional patterns
Primitive types involved	byte, short, int, long, float, double, char, Boolean
Main goal	Make primitive values first-class participants in pattern matching
Key benefit	More expressive type-based branching
Performance benefit	Can avoid unnecessary boxing in suitable code
Related features	Pattern Matching for instanceof, switch expressions, record patterns, sealed types
Main limitation	Still a preview feature in Java 25

2. What problem does it solve?

Traditional Java switch has historically been centered around particular primitive types and reference types with different rules.

Pattern matching has evolved toward:

```
switch (value) {
    case String s -> ...
    case Integer i -> ...
}
```

But this uses **reference types**, so primitive values may need boxing:

```
int
↓
Integer
↓
```

pattern matching

Primitive Types in Patterns moves toward:

int
↓
primitive pattern
↓
match directly

This is particularly useful when a program needs to combine **primitive values, ranges, types, and pattern matching** in one switch.

3. Advantages

Advantage	Explanation	Real-world benefit
Direct primitive pattern matching	Primitive values can participate directly in patterns	Cleaner numeric branching
Less boxing	Avoids unnecessary conversion to wrapper objects in suitable cases	Potentially less allocation/GC pressure
Better switch expressiveness	Numeric types can be integrated into pattern-oriented switches	More readable decision logic
Range-oriented logic	Primitive patterns can be useful with relational cases	Excellent for validation/classification
Better type consistency	Primitive and reference pattern matching become more unified	Easier language model
Cleaner numeric code	Can reduce chains of if/else	More declarative logic
Works with pattern matching	Builds on Java's existing pattern-matching model	Familiar syntax for modern Java developers
Useful for data classification	Values can be categorized directly	Useful in business rules
Potential performance benefits	Primitive operations avoid wrapper overhead	Useful in high-throughput code

Better integration with generic pattern matching	Primitive cases can coexist with other patterns	More expressive switch statements
Reduced manual conversion	Less explicit boxing/unboxing code	Cleaner source
Modern Java direction	Continues Java's pattern-matching evolution	More consistent modern syntax

4. Disadvantages

Disadvantage	Explanation	Impact
Preview feature	Third Preview means the specification can still evolve	Production adoption requires caution
Syntax learning	Developers need to understand primitive pattern semantics	Learning curve
Potential overuse	Simple if conditions can become unnecessarily complicated patterns	Reduced readability
Numeric conversion rules	Primitive widening/narrowing interactions require careful understanding	Possible subtle bugs
Floating-point complexity	float/double have special values such as NaN and signed zero	Edge cases need attention
Not always faster	Avoiding boxing doesn't guarantee measurable performance improvement	Benchmark before optimizing
Older Java compatibility	Code using preview syntax cannot be compiled as ordinary Java 17/21 source	Limits portability
Tooling considerations	IDE/compiler/build tools must support the preview feature	Build configuration required

Pattern complexity	Complex patterns can become difficult to read	Prefer simple cases
Migration risk	Preview syntax can change	Code may need modification in future Java releases

5. When to use Primitive Types in Patterns

Situation	Use?	Reason
Numeric classification	☑ Yes	Natural fit
Range-based business rules	☑ Yes	Makes rules declarative
Parsing numeric data	☑ Yes	Useful for categorization
Validation logic	☑ Yes	Clear value-based branching
switch with multiple numeric cases	☑ Yes	Can improve readability
Mixed primitive/reference pattern matching	☑ Yes	Unified pattern model
High-throughput numeric processing	☑ Potentially	Can avoid unnecessary boxing
Financial/business rules	☑ Potentially	Clear categorization, but validate numeric semantics carefully
Simple equality test	☑ Usually unnecessary	Ordinary if may be clearer
Very simple int switch	☑	Traditional switch may be simpler
Java 17/21 compatibility required	☑ No	Preview feature isn't available
Public library requiring broad Java compatibility	☑ Avoid for now	Preview dependency
Extremely complex pattern	☑ Usually no	Can hurt readability

6. When NOT to use it

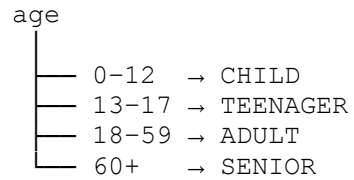
Scenario	Why not?	Better alternative
Simple comparison	Pattern matching adds unnecessary complexity	if
Simple integer switch	Traditional switch is already clear	Switch
Java 21 LTS-only application	Feature isn't part of Java 21	Existing pattern/switch features
Java 17 application	Not available	Traditional if/switch
Public API/library needing stable language features	Preview syntax creates compatibility concerns	Final Java features
Performance-critical code without benchmark evidence	Theoretical optimization isn't enough	Benchmark ordinary vs pattern implementation
Heavy floating-point logic	Special values can complicate semantics	Explicit numeric conditions
Team unfamiliar with preview features	Maintenance can become difficult	Established Java constructs
Code with very complex nested patterns	Can become less readable	Extract methods or use explicit conditions

7. Real-time Use Case 1 — Customer age classification

Suppose an application classifies customers:

0-12 → CHILD
13-17 → TEENAGER
18-59 → ADULT
60+ → SENIOR

Conceptually:



Primitive patterns can make numeric classification more expressive.

This is useful for:

- insurance systems,
- retail pricing,
- eligibility rules,
- education systems,
- access policies.

8. Real-time Use Case 2 — HTTP status classification

An HTTP server receives:

statusCode

You may classify:

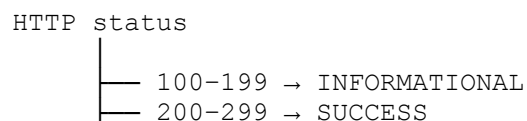
1xx → informational
2xx → success
3xx → redirection
4xx → client error
5xx → server error

Instead of scattering conditions:

```
if (status >= 200 && status < 300) {  
  ...  
} else if (status >= 400 && status < 500) {  
  ...  
}
```

pattern-oriented primitive matching can express the classification more declaratively.

Conceptually:



└─	300-399	→	REDIRECTION
└─	400-499	→	CLIENT_ERROR
└─	500-599	→	SERVER_ERROR

9. Real-time Use Case 3 — Temperature classification

An IoT system receives:

double temperature

and classifies:

< 0 °C	→	FREEZING
0-20 °C	→	COLD
21-30 °C	→	NORMAL
31-40 °C	→	HOT
> 40 °C	→	EXTREME

This is a natural primitive-pattern use case because the value is:

- numeric,
- primitive,
- classified according to ranges.

10. Real-time Use Case 4 — Payment amount rules

Suppose:

paymentAmount

is categorized:

≤ ₹1,000	→	SMALL
₹1,001-₹10,000	→	NORMAL
₹10,001-₹1L	→	LARGE
> ₹1L	→	HIGH_VALUE

A payment system can use such classification for:

- fraud screening,
- approval workflows,
- audit rules,
- risk scoring.

Important: for financial calculations, use appropriate decimal arithmetic such as BigDecimal rather than relying on binary floating-point values for monetary precision. Primitive pattern matching doesn't change that rule.

11. Real-time Use Case 5 — Sensor values

An industrial IoT application receives:

```
int pressure;
```

Classification:

```
pressure < 20      → LOW
20-80              → NORMAL
81-100             → HIGH
> 100              → CRITICAL
```

This can make control logic more declarative:

```
Sensor
  ↓
Primitive pattern
  ↓
LOW / NORMAL / HIGH / CRITICAL
  ↓
Action
```

For safety-critical systems, explicit validation and fail-safe behavior are still required.

12. Real-time Use Case 6 — CPU utilization

A monitoring application receives:

```
int cpuUsage;
```

Business classification:

```
0-49      → NORMAL
50-79     → WARNING
80-94     → HIGH
95-100    → CRITICAL
```

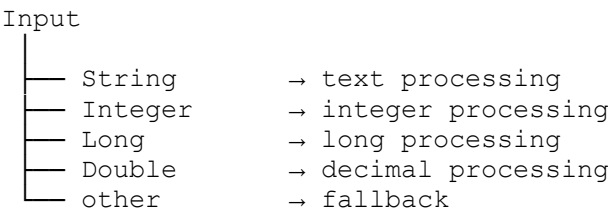
This can be useful in:

- monitoring dashboards,
- autoscaling decisions,
- alerting,
- infrastructure management.

13. Real-time Use Case 7 — Mixed primitive and reference patterns

The broader benefit becomes more apparent when a program needs to process values of different types.

Conceptually:



Primitive pattern support helps reduce the conceptual distinction between:

```
primitive pattern
      vs
reference pattern
```

and contributes to a more unified pattern-matching model.

14. Primitive Patterns vs traditional switch

Aspect	Traditional switch	Primitive Types in Patterns
Integer cases	✓	✓
Character cases	✓	✓
String cases	✓	✓
Numeric patterns	Limited	More expressive
Type patterns	✗	✓
Pattern matching	Limited	✓
Range-oriented patterns	Requires explicit logic	Better suited

Primitive/reference integration	Separate concepts	More unified
Boxing avoidance	Depends	Can avoid unnecessary boxing
Simplicity for basic cases	Excellent	May be unnecessary

15. Primitive Types in Patterns vs if/else

Aspect	if/else	Primitive patterns
Simple condition	Excellent	Good but possibly excessive
Multiple ranges	Can become verbose	More declarative
Complex boolean expression	Excellent	Not always better
Pattern matching	Limited	Excellent
Readability	Good for small logic	Excellent for structured cases
Performance	Usually good	Usually good; benchmark if important
Familiarity	Very high	Requires modern Java knowledge
Java compatibility	Very broad	Requires Java 25 preview
Best use	General conditional logic	Pattern-oriented decision logic

16. Primitive Types in Patterns vs boxing

Traditional approach:

```

int
↓
Integer
↓
pattern matching
↓
unboxing / processing

```

Primitive pattern approach:

int
↓
primitive pattern
↓
processing

Factor	Boxing approach	Primitive pattern
Wrapper object	May be required	Can be avoided
Allocation	Potentially	Lower potential overhead
GC pressure	Potentially higher	Potentially lower
Type conversion	More involved	More direct
Numeric processing	Less direct	More direct
Performance	Depends on workload	Potential advantage
Guaranteed speedup	✗	✗

Don't assume every primitive pattern is automatically faster. The JIT already performs many optimizations, so benchmark real workloads.

17. Floating-point considerations

Primitive pattern matching involving float and double requires special care.

Value	Consideration
Positive numbers	Normal
Negative numbers	Normal
0.0	Normal
-0.0	Can have distinct semantics
NaN	Does not behave like ordinary numeric values
+Infinity	Special value
-Infinity	Special value

--	--

For example:

```
double value
├── normal number
├── +Infinity
├── -Infinity
└── NaN
```

Therefore, financial, scientific, and telemetry applications should explicitly define their handling of special floating-point values.

18. Advantages in high-performance applications

Workload	Potential benefit
Millions of numeric classifications	☑ Potentially useful
High-frequency telemetry	☑ Good candidate
Network protocol processing	☑ Good candidate
Numeric parsers	☑ Good candidate
Data processing pipelines	☑ Good candidate
Simple one-time business rule	☑ Probably insignificant
Heavy object allocation elsewhere	☑ Pattern optimization may not matter
CPU-bound numerical algorithm	☑ Benchmark required

The biggest performance-related advantage is **avoiding unnecessary boxing**, but the primary value of the feature is language expressiveness.

19. Alternate features

Because this is a **language feature**, alternatives depend on what you are trying to accomplish.

Requirement	Alternative	When preferable
-------------	-------------	-----------------

Simple numeric branching	if/else	Small number of conditions
Integer/char switching	Traditional switch	Exact-value cases
String switching	switch	String classification
Object type matching	Pattern matching for instanceof	Reference-type checks
Complex asynchronous decisions	CompletableFuture	Async workflow rather than value matching
Domain classification	enum	Fixed named categories
Complex business rules	Strategy pattern	Rules need independent behavior
Numeric ranges	if/else	Simple range logic
High-performance primitive processing	Direct primitive operations	Maximum control
Data-oriented processing	Arrays / primitive collections	Large numeric datasets

20. enum as an alternative

Sometimes primitive patterns are not the best solution.

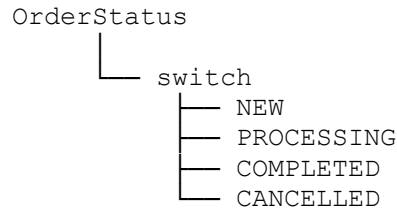
Suppose:

```
1 → NEW
2 → PROCESSING
3 → COMPLETED
4 → CANCELLED
```

Rather than repeatedly pattern matching on integers, convert the value to:

```
enum OrderStatus {
    NEW,
    PROCESSING,
    COMPLETED,
    CANCELLED
}
```

Then:

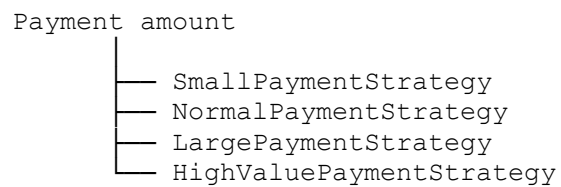


Rule

If the numbers represent **domain concepts**, an enum is often more maintainable than primitive values.

21. Strategy Pattern as an alternative

If every category requires different behavior:



a Strategy pattern may be better than putting a large amount of behavior inside a switch.

Situation	Preferred
Small classification	Primitive pattern
Many independent behaviors	Strategy
Fixed domain states	enum
Simple conditions	if/else
Exact numeric values	switch

22. Compatibility considerations

Java version	Primitive Types in Patterns
Java 17	✗

Java 18	✗
Java 19	✗
Java 20	✗
Java 21	✗
Java 22	Preview evolution
Java 23	Preview evolution
Java 24	Preview evolution
Java 25	Third Preview

Because Java 25 still treats this as a **preview**, compiling and running code that uses the feature requires preview support.

23. Best practices

Best practice	Recommendation
Use patterns when they simplify classification	✓
Keep patterns readable	✓
Prefer primitive values for genuinely numeric data	✓
Avoid unnecessary boxing	✓
Use enum for domain states	✓
Use explicit parameters for business data	✓
Benchmark performance claims	✓
Handle floating-point edge cases	✓
Consider BigDecimal for monetary arithmetic	✓
Avoid complicated nested patterns	✓

Use preview features deliberately	✓
Replace simple if statements with patterns automatically	✗
Use patterns just because they are new	✗

24. Decision matrix

Requirement	Recommendation
Numeric range classification	🔍 Primitive patterns
Numeric type-based matching	🔍 Primitive patterns
Mixed primitive/reference pattern matching	🔍 Primitive patterns
High-frequency numeric classification	🔍 Consider primitive patterns
Simple <code>int == 10</code>	🔍 Traditional if may be clearer
Simple integer switch	🔍 Traditional switch may be clearer
Fixed business states	🔍 enum
Complex behavior per category	🔍 Strategy pattern
Java 17/21 compatibility	🔍 Don't use
Stable production API	🔍 Be cautious due to preview status
Floating-point scientific processing	🔍 Use carefully
Financial calculations	🔍 Use appropriate decimal types
Performance optimization without benchmark	🔍 Don't assume benefit

25. Overall assessment

Category	Rating	Explanation
----------	--------	-------------

Language expressiveness	★★★★★	Major improvement to pattern matching
Numeric classification	★★★★★	Excellent use case
Readability	★★★★☆	Excellent when patterns remain simple
Boxing avoidance	★★★★☆	Can reduce unnecessary wrapper handling
Performance potential	★★★★☆	Useful in suitable workloads
Business-rule suitability	★★★★☆	Good for classification
Complexity	★★★☆☆	Can become complex if overused
Learning curve	★★★☆☆	Requires understanding new pattern semantics
Compatibility	★★☆☆☆	Preview feature in Java 25
Long-term stability	★★★☆☆	Still subject to preview evolution
Simple conditions	★★★☆☆	Traditional if can be clearer
Modern Java applications	★★★★☆	Strong candidate where pattern matching is already used

Final recommendation

Scenario	Recommendation
Classify numeric ranges	🔍 Use
Process primitive values through pattern matching	🔍 Use
Combine primitive and reference patterns	🔍 Use
High-volume numeric classification	🔍 Consider
Telemetry/sensor processing	🔍 Good candidate
HTTP/status/range classification	🔍 Good candidate

Simple equality check	❓ Use if
Simple integer switch	❓ Use traditional switch
Fixed business states	❓ Prefer enum
Complex behavior per case	❓ Consider Strategy
Java 17/21 compatibility required	❓ Don't use
Public library requiring stable syntax	❓ Avoid until final
Financial calculations	❓ Use carefully; pattern matching doesn't solve decimal-precision issues
Floating-point-heavy logic	❓ Explicitly handle NaN, infinities and signed zero

Bottom line

Primitive Types in Patterns extends Java's pattern-matching model from primarily reference-oriented matching toward a more unified model that also understands primitive values.

Its strongest practical use is:

```

Primitive value
    ↓
Pattern matching
    ↓
Classification / validation
    ↓
Business action

```

For example:

```

CPU usage
    ↓
< 50      → NORMAL
50-79     → WARNING
80-94     → HIGH
95-100    → CRITICAL

```

However, **don't replace every if or traditional switch with primitive patterns**. Use them when they make numeric classification and pattern-oriented logic clearer. Since this is the **third preview in Java 25**, teams should also isolate preview-feature usage and avoid making it a hard dependency in libraries that need broad Java-version compatibility.

Java 25 — Vector API (Tenth Incubator)

UniqueClassName: VectorAPI
JEP: 508
Java Version: 25
Year: 2025
Status: Tenth Incubator

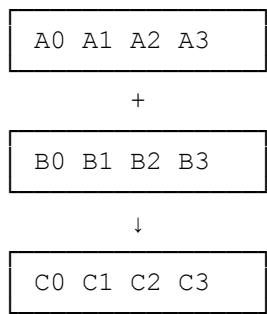
The **Vector API** provides Java developers with a way to express **SIMD (Single Instruction, Multiple Data)** computations. Instead of processing one value at a time, SIMD hardware can process multiple values in parallel using CPU vector instructions.

For example:

Traditional scalar:

A[0] + B[0] → C[0]
A[1] + B[1] → C[1]
A[2] + B[2] → C[2]
A[3] + B[3] → C[3]

Vector:



The JVM can map vector operations to appropriate CPU SIMD instructions where supported.

1. Feature overview

Aspect	Details
Feature	Vector API
UniqueClassName	VectorAPI
JEP	508
Java Version	25

Year	2025
Status	Tenth Incubator
Main package	jdk.incubator.vector
Main purpose	Express SIMD/vector computations in Java
Programming model	Explicit vector operations
Hardware model	CPU SIMD/vector instructions
Typical data	int, long, float, double, etc.
Best workloads	Numeric, scientific, image, signal, ML and data processing
Main benefit	Process multiple independent values simultaneously
Main limitation	Incubator API and workload-dependent performance
Primary alternative	Scalar Java code, JIT auto-vectorization, native SIMD libraries
Important point	It is not GPU programming

2. What is SIMD?

SIMD means:

Single Instruction, Multiple Data

Suppose you need to add 1 million integers.

Scalar processing:

1 + 10

2 + 20

3 + 30

4 + 40

...

A vector-capable CPU may process multiple integers in one operation:

[1, 2, 3, 4]

+

[10, 20, 30, 40]

[11, 22, 33, 44]

Instead of:

1 operation → 1 value

you can potentially get:

1 vector operation → many values

This is why Vector API can be valuable for computationally intensive workloads.

3. Advantages

Advantage	Explanation	Real-world benefit
SIMD parallelism	Processes multiple values with one vector instruction	Higher throughput
High numerical performance	Excellent for repetitive arithmetic	Faster numerical workloads
Pure Java API	SIMD algorithms can be expressed directly in Java	Avoids mandatory native code
Portable vector abstraction	Code targets vector species rather than one exact CPU instruction	Better cross-platform design
CPU-specific optimization	JVM can select suitable hardware instructions	Can exploit available SIMD capabilities
Less JNI dependency	Many SIMD algorithms can remain inside Java	Easier deployment
Good for arrays	Sequential numeric arrays map naturally to SIMD	Fast data processing
Excellent for image processing	Pixels can be processed in parallel	Faster image operations
Excellent for signal processing	Samples often require identical operations	High throughput

Useful for ML workloads	Matrix/vector operations can benefit	Faster inference/preprocessing
Useful for cryptography	Some algorithms contain parallel operations	Potential performance improvement
Better than manually writing CPU intrinsics	Java provides a higher-level abstraction	Easier maintenance
Works with modern JIT	JVM can compile vector operations to hardware instructions	Strong runtime optimization potential
Predictable vector semantics	Programmer explicitly expresses vectorization	More control than relying entirely on auto-vectorization

4. Disadvantages

Disadvantage	Explanation	Impact
Still an incubator	API isn't final Java SE API	Production compatibility concerns
More complex than normal Java	Developers must understand vector concepts	Learning curve
Not automatically faster	Vectorization has overhead and workload requirements	Benchmarking is essential
Data must be suitable	Independent operations benefit most	Branch-heavy code may perform poorly
Memory bandwidth can dominate	CPU may wait for data rather than computation	SIMD may provide little benefit
Alignment/layout issues	Data layout affects efficiency	Poor layouts can reduce benefits
Tail handling	Array length may not exactly match vector width	Additional handling may be required
Hardware differences	SIMD capabilities vary between CPUs	Performance can vary
Debugging complexity	Vector code is less familiar	Maintenance may be harder

Portability isn't identical performance	API portability doesn't guarantee equal speed	Different hardware produces different results
Not a GPU API	Doesn't provide general GPU execution	Wrong choice for GPU-scale workloads
Overkill for small data	Vector setup/loop overhead may dominate	Scalar code may be faster
Branch-heavy algorithms	Divergent operations reduce SIMD effectiveness	Limited benefit
Floating-point semantics	Vectorization can interact with numerical behavior	Numerical testing required
Requires incubator configuration	Special compilation/runtime options are involved	Deployment complexity

5. When to use Vector API

Situation	Use?	Why
Large integer arrays	☑ Yes	Excellent SIMD candidate
Large floating-point arrays	☑ Yes	Parallel arithmetic
Image processing	☑ Yes	Pixels can often be processed independently
Audio processing	☑ Yes	Audio samples are naturally parallel
Signal processing	☑ Yes	Repeated mathematical operations
Scientific computing	☑ Yes	Large numerical datasets
Numerical simulations	☑ Yes	Many independent calculations
ML preprocessing	☑ Yes	Vectorizable numerical operations
ML inference components	☑ Potentially	Matrix/vector calculations
Compression algorithms	☑ Potentially	Some stages are highly parallel

Hashing/cryptographic primitives	❓ Potentially	Some algorithms have SIMD-friendly operations
Large data transformations	❓ Yes	Repetitive element-wise processing
Small collections	❓ Usually no	Overhead may outweigh benefits
Branch-heavy algorithms	❓ Usually no	Poor SIMD utilization
Pointer-heavy object graphs	❓ No	Not vector-friendly
I/O-bound application	❓ No	CPU SIMD isn't the bottleneck
Simple business logic	❓ No	Complexity isn't justified
Database-heavy application	❓ Usually no	Database/network is often bottleneck
GPU workload	❓ No	Use GPU-oriented technology
Java library requiring stable API	❓ Avoid for now	Incubator status

6. Real-time Use Case 1 — Image processing

An image contains millions of pixels.

Suppose each pixel needs:

```
newPixel = oldPixel * brightness
```

Scalar:

```
Pixel 1 → calculate
Pixel 2 → calculate
Pixel 3 → calculate
Pixel 4 → calculate
```

Vector:

P1	P2	P3	P4
----	----	----	----

↓

SIMD operation



R1	R2	R3	R4
----	----	----	----

Applications

- image resizing,
- brightness adjustment,
- contrast adjustment,
- color conversion,
- image filtering,
- computer vision preprocessing.

7. Real-time Use Case 2 — Audio processing

An audio application may process:

48,000 samples/second

Suppose every sample requires:

$\text{output} = \text{input} \times \text{gain}$

Scalar:

```
sample1 × gain
sample2 × gain
sample3 × gain
...
```

Vector:

```
[s1 s2 s3 s4 s5 s6 s7 s8]
      ×
[g  g  g  g  g  g  g  g]
      ↓
[o1 o2 o3 o4 o5 o6 o7 o8]
```

This is an excellent SIMD workload because the same mathematical operation is applied independently to many samples.

8. Real-time Use Case 3 — Financial risk calculations

Consider a risk engine processing millions of values:

portfolio returns
market prices
volatility values
risk factors

For example:

`result[i] = price[i] * quantity[i]`

A vectorized implementation can potentially process multiple records simultaneously.

Suitable areas

- portfolio calculations,
- risk scoring,
- Monte Carlo preprocessing,
- statistical calculations,
- numerical transformations.

Important: financial correctness takes priority over micro-optimization. Use appropriate numeric representations and validate numerical behavior carefully.

9. Real-time Use Case 4 — Machine-learning preprocessing

Suppose an ML pipeline normalizes millions of values:

`normalized = (value - mean) / standardDeviation`

The same calculation is performed repeatedly:

[x1 x2 x3 x4]
↓
subtract mean
↓
divide by stddev
↓
[y1 y2 y3 y4]

This is a classic vectorizable workload.

Potential applications:

- feature normalization,
- image preprocessing,

- signal preprocessing,
 - embedding transformations,
 - numerical inference components.
-

10. Real-time Use Case 5 — Search and filtering

Suppose a large numerical dataset contains:

10 million values

You want to determine which values satisfy:

`value > threshold`

A SIMD implementation can compare multiple values in parallel and generate a mask:

```
Values:  
[12, 85, 33, 91]
```

```
Threshold = 50
```

```
Result:  
[ F,  T,  F,  T ]
```

This is useful for:

- analytics,
 - telemetry processing,
 - data filtering,
 - time-series systems,
 - scientific datasets.
-

11. Real-time Use Case 6 — Signal processing

A signal-processing system might perform:

```
output[i] =  
    input[i] * coefficient[i];
```

for millions of samples.

Examples:

- radar processing,
- sensor analysis,
- vibration analysis,
- telecommunications,
- industrial monitoring.

The repetitive numerical nature makes these excellent Vector API candidates.

12. Real-time Use Case 7 — Cryptographic algorithms

Some cryptographic operations contain parallelizable transformations.

For example:

```
Block 1  ┌
Block 2  │ → parallel transformation
Block 3  │
Block 4  └
```

Vector operations can potentially accelerate suitable cryptographic primitives.

However:

Don't implement production cryptography merely to experiment with Vector API.

Use established cryptographic libraries and rely on well-tested implementations where possible.

13. Basic conceptual Java example

The Vector API uses concepts such as VectorSpecies and vector types.

A simplified structure looks like:

```
import jdk.incubator.vector.*;

class VectorExample {

    static final VectorSpecies<Integer> SPECIES =
        IntVector.SPECIES_PREFERRED;

    static void add(int[] a, int[] b, int[] result) {

        int i = 0;
```

```

for (; i < SPECIES.loopBound(a.length);
    i += SPECIES.length()) {

    IntVector va =
        IntVector.fromArray(SPECIES, a, i);

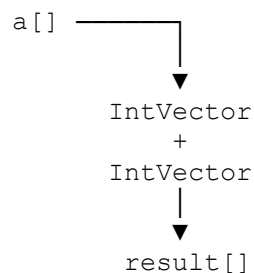
    IntVector vb =
        IntVector.fromArray(SPECIES, b, i);

    va.add(vb).intoArray(result, i);
}

for (; i < a.length; i++) {
    result[i] = a[i] + b[i];
}
}

```

Conceptually:



The final scalar loop handles the **tail** when the array length isn't an exact multiple of the vector width.

14. Why SPECIES_PREFERRED matters

One of the important Vector API concepts is the **species**.

Instead of hard-coding:

"Process exactly 8 integers"

the code can use a preferred species:

`IntVector.SPECIES_PREFERRED`

This allows the runtime to choose an appropriate vector shape for the target hardware.

Conceptually:

CPU A → preferred vector width

CPU B → preferred vector width

CPU C → preferred vector width

The Java algorithm remains conceptually the same.

15. Vector API vs ordinary Java loop

Aspect	Ordinary loop	Vector API
Code complexity	☐ Low	☐ Higher
Readability	☐ Excellent	☐ Moderate
SIMD control	☐ Indirect	☐ Explicit
Portability	☐ Excellent	☐ Good API portability
Hardware exploitation	JVM may auto-vectorize	Explicit vector operations
Performance ceiling	Potentially high	Potentially very high
Developer effort	Low	Higher
Best for	General application code	Performance-critical numerical code
Tail handling	Simple	Must account for vector boundaries
Benchmarking required	Sometimes	Yes

16. Vector API vs JIT auto-vectorization

This is one of the most important alternatives.

Modern HotSpot can sometimes automatically vectorize ordinary loops.

For example:

```
for (int i = 0; i < length; i++) {  
    result[i] = a[i] + b[i];  
}
```

The JIT may recognize that the loop is vectorizable.

Aspect	JIT auto-vectorization	Vector API
Developer code	Simple	More explicit
SIMD control	Limited	High
Compiler decides	Yes	Programmer expresses vector operation
Code complexity	Low	Higher
Performance portability	Good	Good
Hardware-specific behavior	JVM handles it	JVM maps vector API
Best starting point	Yes	After profiling
Best for maximum SIMD control	Limited	Excellent

Practical rule

Start with:

Simple Java loop
↓
Benchmark/profile
↓
Check JIT optimization
↓
If insufficient
↓
Consider Vector API

Don't automatically rewrite every numeric loop using Vector API.

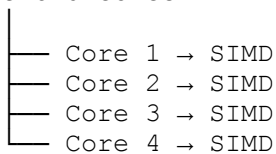
17. Vector API vs parallel streams

Aspect	Vector API	Parallel Streams
--------	------------	------------------

Parallelism type	SIMD	Multiple threads
Data level	Within CPU instruction	Across CPU cores
Thread creation	No application-level parallel threads required	Uses parallel execution
Best workload	Element-wise numerical operations	Larger independent data tasks
Programming complexity	Higher	Lower
Fine-grained numeric operations	Excellent	Often inefficient
CPU core scaling	SIMD lanes	Multiple cores
Memory overhead	Low	Higher
Suitable for image/audio processing	Excellent	Sometimes
Suitable for coarse-grained computations	Good	Excellent

They can conceptually complement each other:

Multiple CPU cores



But combining parallelism levels should be done carefully and benchmarked.

18. Vector API vs GPU

Aspect	Vector API	GPU
Hardware	CPU	GPU
SIMD/SIMT	CPU SIMD	GPU-oriented parallel execution
Data transfer	Usually no CPU↔GPU	May require transfer

	transfer	
Small numerical operations	Good	Often inefficient
Huge parallel workloads	Good	Excellent
Java integration	Native Java incubator API	Requires GPU framework/library
Hardware availability	Broad	Requires suitable GPU
Programming complexity	Moderate	Usually higher
Best for	CPU vectorization	Massive parallel computation

If the workload is:

100 million independent mathematical operations

a GPU may be attractive.

If it is:

moderate numerical workload already resident in CPU memory

Vector API may be a better fit.

19. Vector API vs JNI/native SIMD

Aspect	Vector API	JNI/native SIMD
Language	Java	C/C++/native
Deployment	Simpler	Native binaries required
JNI overhead	Avoided	Possible
Hardware control	High	Very high
Portability	Better	Lower
Maintenance	Easier	More difficult
Memory safety	Java-managed	Native risks

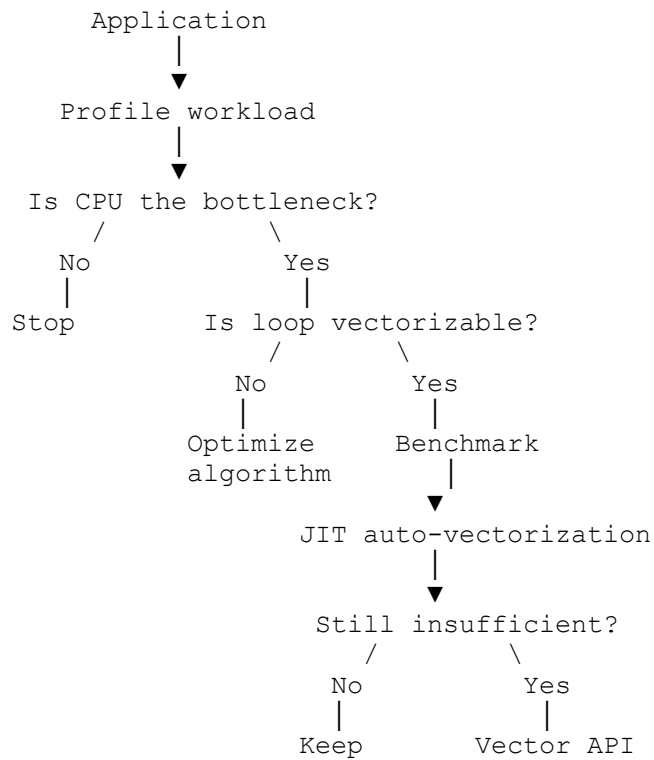
Existing native library integration	Not necessary	Useful
Maximum specialized optimization	Good	Potentially highest
Best choice for new Java SIMD code	Strong candidate	Only when justified

20. When NOT to use Vector API

Scenario	Why not?	Better choice
CRUD business application	SIMD isn't relevant	Normal Java
REST API mostly waiting on network	CPU isn't bottleneck	Normal concurrency/virtual threads
Database-heavy application	Database latency dominates	Optimize DB/query architecture
Small arrays	Vector overhead may dominate	Scalar loop
Complex branching	Poor SIMD utilization	Normal Java control flow
Object-heavy data	SIMD works best on primitive data	Normal object processing
Frequent random memory access	Poor vector efficiency	Data structure/layout optimization
Need GPU acceleration	CPU SIMD isn't sufficient	GPU framework
Java library needs stable API	Incubator status	Wait/use stable alternative
Algorithm isn't vectorizable	No meaningful SIMD benefit	Scalar/JIT-optimized code
Performance hasn't been measured	Optimization may be unnecessary	Profile first

21. Performance decision process

A good production workflow is:



This avoids using Vector API simply because it is available.

22. Best real-world candidates

Workload	Vectorization potential
Image filtering	★★★★★
Audio processing	★★★★★
Signal processing	★★★★★
Numerical simulation	★★★★★
Large matrix/vector operations	★★★★★
ML preprocessing	★★★★★
Statistical calculations	★★★★☆

Compression	★★★★☆
Cryptographic primitives	★★★★☆
Large array transformations	★★★★★
Database CRUD	☆☆☆☆☆
Web request handling	☆☆☆☆☆
Business workflows	☆☆☆☆☆
Object graph traversal	☆☆☆☆☆
I/O-heavy applications	☆☆☆☆☆

23. Alternate features

Requirement	Alternative	When preferable
Simple numerical loop	Ordinary Java loop	Start here
JVM can optimize loop	JIT auto-vectorization	Best low-complexity option
Multi-core parallel processing	ForkJoinPool / parallel streams	Coarse-grained parallel work
Lightweight I/O concurrency	Virtual Threads	I/O-bound workloads
GPU computation	GPU framework/library	Massive parallel numerical workloads
Native SIMD	JNI/Panama/native library	Existing optimized native implementation
Matrix computation	Specialized math library	Complex linear algebra
ML inference	ONNX Runtime/native ML framework	Full ML workloads

Image processing	Specialized image library	Production image operations
Signal processing	Specialized DSP library	Advanced DSP algorithms

24. Best practice: start with ordinary Java

Suppose you have:

```
for (int i = 0; i < data.length; i++) {
    data[i] *= 2;
}
```

Don't immediately rewrite it as Vector API code.

Use:

1. Write clear scalar Java
2. Test correctness
3. Profile
4. Benchmark
5. Determine whether JIT vectorizes it
6. Only then evaluate Vector API

This is particularly important because modern JVMs are already very sophisticated at optimizing simple loops.

25. Vector API and data layout

SIMD performance depends heavily on data layout.

Good:

```
float[] temperatures
float[] pressures
float[] humidity
```

Potentially less SIMD-friendly:

```
Sensor {
    float temperature;
    float pressure;
    float humidity;
}
```

Sensor[] sensors;

The first arrangement is often easier to process as contiguous primitive arrays.

This leads to an important optimization principle:

Data-oriented design can be as important as the vector instruction itself.

26. Important performance factors

Factor	Impact
Vector width	More elements per vector can increase throughput
CPU SIMD capability	Determines available hardware instructions
Memory bandwidth	Can become the bottleneck
Cache locality	Very important
Data alignment/layout	Can affect efficiency
Branching	Can reduce SIMD benefits
Array size	Large datasets usually benefit more
Arithmetic intensity	More computation per byte can favor SIMD
Tail elements	Require handling
JIT compilation	Can influence final performance
Garbage generation	Avoid unnecessary temporary objects
Benchmark quality	Essential for valid conclusions

27. Overall assessment

Category	Rating	Explanation
Numerical performance	★★★★★	Excellent

potential		
SIMD control	★★★★★	Excellent
Image processing	★★★★★	Excellent
Signal processing	★★★★★	Excellent
Scientific computing	★★★★★	Excellent
ML preprocessing	★★★★★	Excellent
Developer simplicity	★★★★☆	More complex than scalar Java
Portability	★★★★☆	API is portable; performance varies by CPU
Memory efficiency	★★★★☆	Can be excellent with good data layout
General business application usefulness	★★☆☆☆	Usually unnecessary
I/O-bound application usefulness	★☆☆☆☆	Usually irrelevant
API maturity	★★☆☆☆	Tenth Incubator
Performance predictability	★★☆☆☆	Benchmark hardware/workload
Native-code replacement potential	★★★★☆	Strong in many SIMD workloads

28. Final decision table

Scenario	Recommendation
Large numerical arrays	🔍 Use/consider Vector API
Image processing	🔍 Excellent candidate
Audio processing	🔍 Excellent candidate
Signal processing	🔍 Excellent candidate

Scientific computing	🔍 Excellent candidate
ML preprocessing	🔍 Strong candidate
Large statistical calculations	🔍 Good candidate
Compression algorithms	🔍 Potentially useful
Cryptographic primitives	🔍 Use established implementations and benchmark
Simple array loop	🔍 Try ordinary Java/JIT first
Small dataset	🔍 Usually don't use
Branch-heavy algorithm	🔍 Usually don't use
Object-heavy processing	🔍 Usually don't use
I/O-bound service	🔍 Don't use for I/O optimization
CRUD application	🔍 Don't use
GPU-scale workload	🔍 Consider GPU technology
Need stable Java SE API	🔍 Avoid until Vector API becomes final
Performance-critical CPU code	🔍 Benchmark Vector API against scalar/JIT alternatives

Bottom line

Vector API is a specialized performance feature, not a general application-development feature.

Its sweet spot is:

```

Large primitive dataset
+
Same operation on every element
+
CPU-bound workload
+
Good memory locality
↓
SIMD
↓
Potentially much higher throughput

```

For example:

```
[1, 2, 3, 4, 5, 6, 7, 8]
      ×
[2, 2, 2, 2, 2, 2, 2, 2]
      ↓
[2, 4, 6, 8, 10, 12, 14, 16]
```

The recommended progression for production Java is:

ordinary Java loop → profile → let HotSpot auto-vectorize → benchmark → use Vector API only when it provides a demonstrated benefit.

And because **Java 25's Vector API is still the Tenth Incubator**, it is best treated as an advanced performance tool rather than a default programming technique.

Java 25 — JFR CPU-Time Profiling

UniqueClassName: JFRCpuTimeProfiling

JEP: 509

Java Version: 25

Year: 2025

Status: Final

JFR CPU-Time Profiling adds **CPU-time profiling support to Java Flight Recorder (JFR)**. Instead of measuring only elapsed/wall-clock time, it can help identify **how much actual CPU time threads consume**.

This distinction is particularly valuable in applications with a mixture of:

- CPU-bound work,
- blocking I/O,
- waiting,
- sleeping,
- synchronization,
- virtual threads.

1. Feature overview

Aspect	Details
Feature	JFR CPU-Time Profiling
UniqueClassName	JFRCpuTimeProfiling

JEP	509
Java Version	25
Year	2025
Status	Final
Main technology	Java Flight Recorder (JFR)
Primary purpose	Measure/profile CPU time consumed by threads
Main distinction	CPU time vs elapsed/wall-clock time
Particularly useful for	CPU utilization analysis
Works with	Platform threads and virtual threads
Main benefit	Identify where actual CPU resources are being consumed
Complementary technology	JFR events, JMC, JVM profiling
Typical usage	Production performance troubleshooting
Main limitation	CPU-time profiling has measurement overhead and isn't needed for every workload

2. CPU time vs wall-clock time

This distinction is fundamental.

Suppose a REST request takes:

1000 ms

That does **not** mean the CPU worked for 1000 ms.

The request might spend:

CPU computation	100 ms
Database waiting	400 ms
Network waiting	300 ms
Lock waiting	150 ms
Other	50 ms

Total

1000 ms

So:

Metric	Value
Wall-clock time	1000 ms
Actual CPU time	~100 ms
Waiting time	~900 ms

CPU-time profiling helps answer:

"Where did my application's CPU actually go?"

rather than simply:

"Where did my application spend elapsed time?"

3. Advantages

Advantage	Explanation	Real-world benefit
Measures CPU consumption	Shows actual CPU time rather than only elapsed time	Better CPU bottleneck identification
Distinguishes CPU vs waiting	Helps separate computation from blocking	More accurate performance diagnosis
Production profiling	JFR is designed for low-overhead runtime observability	Useful for live systems
Method-level analysis	Can identify CPU-intensive methods	Find optimization targets
Thread-level visibility	Shows which threads consume CPU	Detect hot threads
Virtual-thread support	Particularly useful when many virtual threads exist	Better understanding of CPU usage in modern Java

JFR integration	Combines CPU information with other JVM events	Richer diagnosis
Low operational friction	JFR is built into the JDK	No separate native profiler necessarily required
Continuous profiling possibilities	Can be used during production troubleshooting	Helps catch intermittent problems
Better resource attribution	Helps identify CPU-heavy operations	Supports capacity planning
Supports performance investigations	Correlate CPU with GC, allocation, locks and other events	More complete root-cause analysis
Useful for optimization	Shows whether optimization should target CPU or waiting	Prevents optimizing the wrong component
Useful for capacity planning	CPU demand can be measured more directly	Better infrastructure sizing
Helps detect CPU hotspots	Finds unexpectedly expensive code paths	Faster troubleshooting

4. Disadvantages

Disadvantage	Explanation	Impact
Profiling overhead	Measuring CPU time isn't completely free	Some runtime overhead
Not necessary for every application	Simple applications may not need it	Additional complexity
CPU time isn't wall time	A CPU-heavy method may not explain overall latency	Need both perspectives
Doesn't automatically find root cause	Identifies CPU consumption but not necessarily why code is expensive	Requires further investigation
Sampling/measurement	Profiling results need	Misinterpretation is possible

interpretation	expertise	
Production configuration matters	Aggressive profiling may add more overhead	Need appropriate settings
Doesn't solve performance problems	It only provides measurement	Developer still needs optimization
CPU may not be bottleneck	Database/network/GC could dominate	CPU profiling can be irrelevant
High-core systems complicate interpretation	CPU utilization must be considered relative to available cores	Requires capacity context
Virtual-thread workloads can be complex	Large numbers of tasks make analysis more involved	Requires good event/context interpretation
Short-lived processes	Very short applications may not provide useful profiling data	Limited diagnostic value
Security/privacy considerations	Profiling may expose application execution details	Protect recordings appropriately

5. When to use JFR CPU-Time Profiling

Situation	Use?	Why
CPU utilization unexpectedly high	☒ Yes	Find CPU-consuming code
Application response time is high and CPU is also high	☒ Yes	Determine CPU hotspots
Virtual-thread application has high CPU	☒ Yes	Distinguish CPU work from waiting
Production performance investigation	☒ Yes	JFR is suitable for runtime diagnostics
Capacity planning	☒ Yes	Measure CPU demand

After a deployment CPU increased	☒ Yes	Compare profiling data
One service uses much more CPU than others	☒ Yes	Identify hot code paths
Suspected inefficient algorithm	☒ Yes	Confirm CPU cost
Unexpected CPU spikes	☒ Yes	Investigate hotspots
Benchmarking optimization	☒ Yes	Compare CPU consumption
Database/network latency is the issue	☒ Usually not first	Profile I/O/waiting instead
Tiny command-line program	☒ Usually no	Profiling overhead isn't worthwhile
CPU is clearly idle	☒ No	CPU profiling won't solve an I/O problem
Simple application with no performance issue	☒ No	No reason to profile
Real-time hard-latency environment	☒ Carefully	Profiling overhead must be evaluated

6. When NOT to use it

Scenario	Why not?	Better approach
Database is clearly slow	CPU isn't primary bottleneck	Database/query profiling
Remote API is slow	Network waiting dominates	HTTP/network tracing
GC pauses dominate	CPU-time profiling isn't sufficient	JFR GC analysis
Memory leak suspected	CPU isn't the main measurement	Heap/allocations analysis
Deadlock suspected	CPU profile won't explain	Thread/lock analysis

	waiting fully	
Simple low-traffic service	Little value	Basic monitoring
No performance problem	Profiling is unnecessary	Standard observability
Startup-only issue	Continuous CPU profiling may not be appropriate	Startup/JFR recording
Need deep native CPU analysis	JFR may not provide every native detail needed	OS/native profiler
GPU bottleneck	CPU-time data doesn't explain GPU workload	GPU profiling

7. Real-time Use Case 1 — High CPU in a REST service

Suppose:

Production API

CPU utilization = 95%

Response time = 800 ms

Traditional monitoring tells you:

CPU = 95%

But not necessarily:

Which Java code consumes the CPU?

JFR CPU-time profiling can help identify:

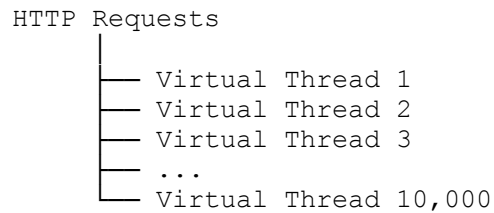
95% CPU

—	JSON processing	25%
—	encryption	20%
—	business calculation	35%
—	logging	10%
—	other	10%

Now the team knows where to investigate.

8. Real-time Use Case 2 — Virtual-thread application

Suppose an application uses thousands of virtual threads:

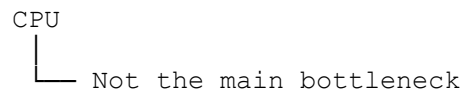


Most threads may spend their time waiting for:

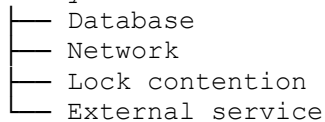
- databases,
- HTTP services,
- files,
- queues.

Suppose overall latency is high but CPU utilization is only 30%.

That suggests:



Likely:



CPU-time profiling helps make that distinction.

9. Real-time Use Case 3 — CPU regression after deployment

Suppose version 1:

CPU = 40%

Version 2:

CPU = 75%

The application functionality is unchanged.

JFR recordings can help identify a change such as:

Version 1:
JSON serialization → 10% CPU

Version 2:
JSON serialization → 28% CPU

or:

Version 1:
Database processing → 15%

Version 2:
Database processing → 15%

New algorithm → 30%

This helps narrow the regression.

10. Real-time Use Case 4 — Inefficient JSON processing

Imagine a service receives:

1 million JSON documents/hour

CPU becomes saturated.

Profiling reveals:

JSON parsing	35%
Object mapping	25%
Business logic	15%
Logging	10%
Other	15%

The team can then investigate:

- repeated serialization,
- unnecessary object creation,
- inefficient parsing,
- excessive logging,
- redundant transformations.

Without CPU-time information, developers might incorrectly optimize database access.

11. Real-time Use Case 5 — Cryptographic workload

Suppose an authentication service performs:

TLS
JWT signing
Password hashing
Encryption

CPU usage suddenly increases.

CPU-time profiling may show:

```
Cryptographic operations
      ↓
    60% CPU
```

This tells the team the bottleneck is computational rather than network latency.

Potential next steps:

- benchmark algorithms,
- inspect configuration,
- investigate excessive cryptographic operations,
- evaluate hardware acceleration,
- cache appropriate non-secret computations.

12. Real-time Use Case 6 — Image/video processing

An image-processing service might receive:

```
Upload image
      ↓
    Decode
      ↓
    Resize
      ↓
    Transform
      ↓
    Compress
```

CPU profiling could reveal:

Decode	10%
Resize	40%

Transform	15%
Compress	30%
Other	5%

The team now knows that optimizing the resize algorithm is potentially much more valuable than optimizing other stages.

13. Real-time Use Case 7 — Algorithmic inefficiency

Suppose an application processes:

100,000 records

and CPU usage unexpectedly reaches 100%.

Profiling shows:

calculateScore() → 70% CPU

Code inspection reveals an inefficient algorithm:

$O(n^2)$

instead of something closer to:

$O(n)$

JFR doesn't automatically redesign the algorithm, but it can help identify where CPU resources are being consumed.

14. Real-time Use Case 8 — CPU vs I/O diagnosis

Suppose an API has:

Response time = 2 seconds

CPU utilization = 20%

Developers might initially suspect slow Java code.

CPU-time profiling shows:

Actual CPU work = 100 ms

Waiting = ~1900 ms

This strongly suggests looking at:

- database latency,
- network calls,
- locks,
- external APIs,
- filesystem operations.

This is one of the most valuable uses of CPU-time profiling:

It helps prevent optimizing CPU code when CPU isn't actually the problem.

15. CPU time vs wall-clock time

Situation	Wall-clock time	CPU time	Interpretation
Pure CPU calculation	500 ms	~500 ms	CPU-bound
Network request	500 ms	~10 ms	Mostly waiting
Database query	500 ms	~20 ms	Mostly waiting
Lock contention	500 ms	~30 ms	Mostly waiting
Thread sleeping	500 ms	~0 ms	No CPU work
CPU + I/O	500 ms	~200 ms	Mixed workload
GC-intensive processing	High	High	CPU/GC investigation needed

16. JFR CPU-Time Profiling vs traditional CPU monitoring

Aspect	CPU monitoring	JFR CPU-time profiling
CPU percentage	✓	✓
Process-level CPU	✓	✓
JVM-level context	Limited	Excellent
Java method information	✗ / limited	Yes

Thread information	Limited	Yes
JVM event correlation	✗	Excellent
GC correlation	Limited	Yes
Allocation correlation	Limited	Yes
Lock correlation	Limited	Yes
Root-cause investigation	Limited	Strong
Production diagnostics	Good	Excellent

17. JFR CPU-Time Profiling vs JFR traditional sampling

Aspect	Traditional JFR CPU profiling	CPU-time profiling
Measures execution activity	✓	✓
Uses JFR	✓	✓
Wall-clock perspective	Strong	Less central
CPU consumption perspective	Good	Stronger
Distinguishes CPU vs waiting	Limited/indirect	Better
CPU attribution	Good	More direct
Useful for CPU capacity planning	Good	Excellent
Useful for latency analysis	Excellent	Good
Best question	"What is running?"	"Where is CPU time actually being consumed?"

18. JFR CPU-Time Profiling vs async-profiler

async-profiler is a popular external profiler for Java and native workloads.

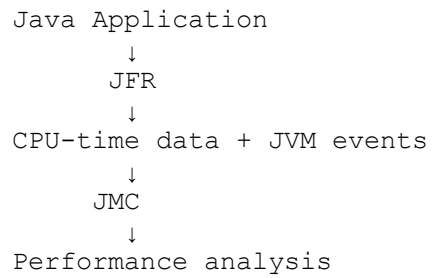
Aspect	JFR CPU-Time Profiling	async-profiler
Included with JDK	Yes	No
Installation	Minimal	Additional tool
JFR integration	Native	Can integrate/export
CPU profiling	✓	Excellent
Native profiling	More limited	Strong
Flame graphs	Via tooling/workflow	Excellent
Production use	Excellent	Excellent
Operational simplicity	Excellent	Good
Deep native analysis	Limited	Excellent
JVM event correlation	Excellent	Good
Best for	JVM-integrated diagnostics	Deep CPU/native profiling

19. JFR CPU-Time Profiling vs Java Mission Control

Java Mission Control (**JMC**) is primarily a **JFR analysis tool**, not a replacement for the JFR recording mechanism.

Component	Role
JFR	Records JVM/application events
CPU-time profiling	Provides CPU-time information
JMC	Analyzes JFR recordings
Application	Produces workload

Conceptually:



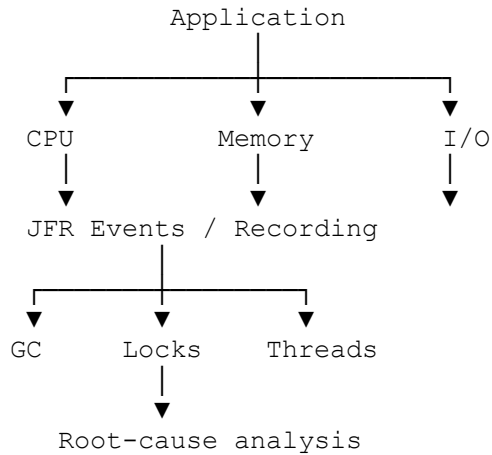
20. Alternate features

Requirement	Alternative
Basic CPU utilization	OS tools such as top, htop, Windows Performance Monitor
JVM CPU hotspot investigation	JFR
Deep CPU profiling	async-profiler
Native CPU profiling	perf / async-profiler / native profilers
Allocation investigation	JFR allocation events
GC investigation	JFR GC events
Lock contention	JFR monitor/Java-thread events
Memory leak investigation	Heap dump + JFR
Distributed request latency	OpenTelemetry/tracing
Database bottleneck	Database profiler/APM
End-to-end application monitoring	APM platform
Low-level Linux CPU analysis	perf
Simple production monitoring	Metrics + JFR when needed

21. When JFR is particularly powerful

The real strength isn't CPU time alone.

JFR can correlate CPU activity with:



For example:

CPU spike
↓
JFR
↓
CPU-heavy method
↓
Large allocations
↓
Frequent GC
↓
Root cause identified

This is much more useful than looking at CPU percentage alone.

22. Performance overhead considerations

Profiling intensity	Typical consideration
Low-overhead JFR configuration	☑ Good for continuous/regular monitoring
Moderate profiling	☑ Good for investigations
Aggressive profiling	☑ Evaluate overhead
Very detailed event configuration	☑ Benchmark overhead
Latency-critical application	☑ Validate carefully
Short-lived application	☑ Recording may have limited value

--	--

The correct approach is:

Use the least intrusive profiling configuration that answers the performance question.

23. Recommended production workflow

Step	Action
1	Detect abnormal CPU utilization
2	Check whether CPU is actually the bottleneck
3	Start/inspect a JFR recording
4	Analyze CPU-time-consuming threads/methods
5	Correlate with allocations
6	Correlate with GC
7	Correlate with locks/waits
8	Identify top CPU-consuming operations
9	Optimize only the significant hotspots
10	Benchmark the change
11	Compare CPU time before/after
12	Validate application latency and throughput

24. Example diagnostic workflow

Suppose:

CPU = 90%

JFR analysis:

```

90% CPU
|
└─ calculateRisk()    35%
```

└─ JSON parsing	25%
└─ GC	15%
└─ logging	10%
└─ other	5%

Now investigate:

```
calculateRisk()
  ↓
O(n²) algorithm
  ↓
Optimize algorithm
  ↓
CPU decreases
```

But suppose instead:

```
CPU = 20%
Latency = 2 seconds
```

JFR shows:

```
CPU work = 150 ms
waiting = 1850 ms
```

Then CPU optimization is probably the wrong first move.

25. Overall assessment

Category	Rating	Explanation
CPU hotspot detection	★★★★★	Excellent
Production diagnostics	★★★★★	Strong JFR capability
JVM integration	★★★★★	Native JDK technology
CPU vs waiting analysis	★★★★★	Major value
Virtual-thread applications	★★★★★	Particularly useful
Root-cause analysis	★★★★★	Excellent when combined with other JFR events
Ease of deployment	★★★★★	Included in JDK

Low overhead	★★★★☆	Designed for low-overhead recording, but configuration matters
Native-code analysis	★★★☆☆	External profilers may be stronger
Database diagnosis	★★☆☆☆	Need database-specific tools
Network diagnosis	★★☆☆☆	Need tracing/network observability
Memory-leak diagnosis	★★★☆☆	JFR helps, but heap dumps may be needed
General application development	★★☆☆☆	It's a diagnostic tool, not application functionality

26. Final decision table

Scenario	Recommendation
Production CPU spike	🔍 Use JFR CPU-Time Profiling
High JVM CPU usage	🔍 Use it
CPU regression after deployment	🔍 Use it
CPU-heavy algorithm	🔍 Use it
Virtual-thread application	🔍 Strong candidate
Capacity planning	🔍 Use it
Performance optimization	🔍 Use it
Need to distinguish CPU from waiting	🔍 Excellent choice
Database is slow	🔍 JFR can help, but use DB profiling too
Network/API latency	🔍 Combine with tracing
GC problem	🔍 Use JFR GC analysis
Memory leak	🔍 Use JFR + heap analysis
Deep native CPU issue	🔍 Consider async-profiler/perf
GPU bottleneck	🔍 Use GPU profiling

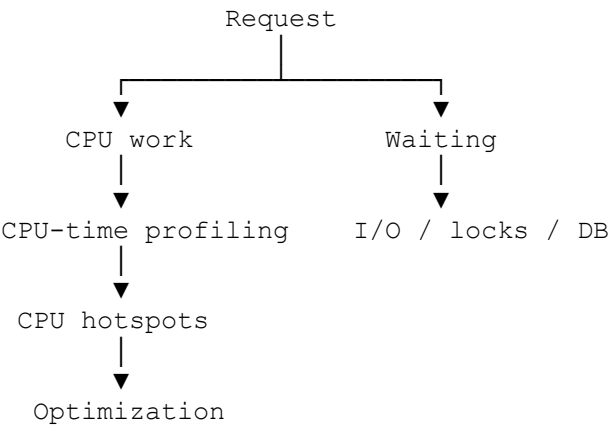
No performance problem	☒ Don't profile unnecessarily
Tiny short-lived application	☒ Usually unnecessary

Bottom line

JFR CPU-Time Profiling is most useful when you need to answer:

"Which parts of my Java application are actually consuming CPU, rather than merely taking elapsed time?"

The distinction is especially valuable in modern Java applications:



Its biggest advantage is that it is **part of the JDK/JFR ecosystem**, allowing CPU-time information to be analyzed alongside GC, allocation, thread, lock, and other JVM events.

Recommended rule: use JFR CPU-time profiling when **CPU consumption is suspected or needs to be quantified**. Don't use it as the first tool for an application whose real bottleneck is clearly a database, network, external service, or other I/O resource.

Java 25 — Key Derivation Function API

Aspect	Details
Feature	Key Derivation Function API
UniqueClassName	KeyDerivationFunctionAPI

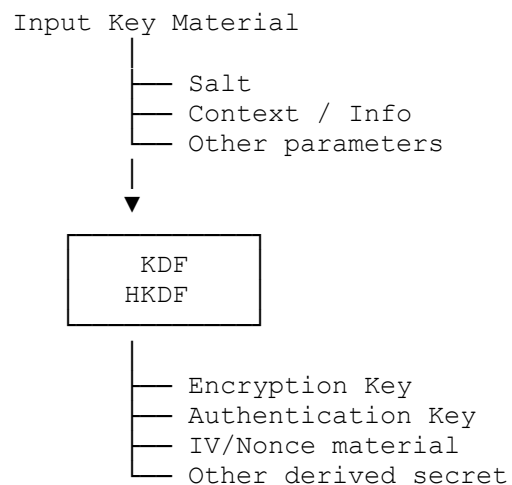
JEP	JEP 510
Java Version	25
Year	2025
Status	Final
Main API	javax.crypto.KDF
Related APIs	KDFParameters, KDFSpi, HKDFParameterSpec
Primary algorithm in JDK	HKDF, including HKDF-SHA256
Purpose	Derive cryptographically strong keys/data from existing keying material and optional context such as salt/info
Main benefit	Standardizes KDF usage in Java instead of requiring application-specific or third-party KDF implementations

JEP 510 finalized the KDF API that was introduced as a preview in Java 24. It is designed for algorithms such as HKDF and for cryptographic protocols including TLS 1.3, HPKE and KEM-based schemes. ([OpenJDK Bug System](#))

1. What is a Key Derivation Function?

A **KDF** takes existing secret/keying material and derives one or more new cryptographic keys.

Conceptually:



Java 25 provides:

```
KDF kdf = KDF.getInstance("HKDF-SHA256");
```

and can derive either a `SecretKey` or raw data using `deriveKey()` and `deriveData()`. ([Oracle Docs](#))

2. Basic Java 25 Example

A simplified HKDF example:

```
import javax.crypto.KDF;
import javax.crypto.SecretKey;
import javax.crypto.spec.HKDFParameterSpec;

byte[] ikm = ...;
byte[] salt = ...;
byte[] info = "payment-service".getBytes();

KDF kdf = KDF.getInstance("HKDF-SHA256");

HKDFParameterSpec spec =
    HKDFParameterSpec.ofExtract()
        .addIKM(ikm)
        .addSalt(salt)
        .thenExpand(info, 32);

SecretKey encryptionKey =
    kdf.deriveKey("AES", spec);
```

The API supports HKDF's Extract, Expand, and Extract-then-Expand operations. ([Oracle Docs](#))

3. Advantages

Advantage	Explanation	Real-world benefit
Standard JDK API	KDF functionality is available through <code>javax.crypto.KDF</code>	Less need for custom KDF implementations
Cryptographically appropriate	Designed specifically for key derivation rather than ordinary hashing	Reduces misuse of generic hashing for key derivation
HKDF support	JDK includes HKDF implementations	Useful for modern cryptographic protocols

Returns SecretKey	deriveKey() can directly produce a SecretKey	Convenient for AES and other cryptographic APIs
Raw-data derivation	deriveData() returns derived bytes	Useful when protocol-specific key material is required
Salt support	KDF parameters can incorporate salt	Improves key separation and cryptographic robustness
Context binding	HKDF info can distinguish different purposes	Prevents accidentally using the same derived key for unrelated purposes
Provider architecture	KDF implementations can be supplied by security providers	Allows Java/native/provider-specific implementations
Hardware/provider integration	Provider selection can account for key material that may reside on hardware	Useful with HSM/PKCS#11-style environments
Non-extractable key support	HKDF parameters can work with secret keys that cannot simply be converted into byte arrays	Important for protected key material
Protocol integration	Designed with TLS 1.3, HPKE and KEM use cases in mind	Useful for modern secure communication systems
Type safety	KDFParameters provides a common parameter abstraction	Cleaner provider/API integration

Java's API explicitly supports both SecretKey results and raw derived data, while the HKDF parameter API supports composing input key material from multiple sources, including non-extractable secret keys. ([Oracle Docs](#))

4. Disadvantages

Disadvantage	Explanation	Impact
Cryptography knowledge required	The API does not automatically choose correct protocol	Incorrect salt/info/output-length choices can still create security problems

	parameters for your application	
Not password hashing by itself	HKDF is not intended to replace password-hardening algorithms such as Argon2id	Don't use HKDF directly for storing user passwords
Algorithm/provider availability	An application may encounter <code>NoSuchAlgorithmException</code> if a requested KDF isn't provided	Deployment/provider configuration matters
API complexity	Extract/Expand/parameter APIs can initially be unfamiliar	Higher learning curve
Not a key-storage solution	KDF derives keys but does not manage their lifecycle or secure storage	You still need a key-management strategy
Not encryption	KDF itself doesn't encrypt application data	It must normally be combined with AES-GCM, ChaCha20-Poly1305, etc.
Not authentication	Derivation doesn't authenticate users or messages	Authentication must be provided separately
Thread-safety consideration	KDF instances are not generally thread-safe unless documented otherwise	Don't blindly share one instance across concurrent threads
Potential interoperability issues	Both sides must use identical KDF algorithm and parameters	Protocol design must precisely define derivation parameters
Java 25 dependency	Applications targeting older Java versions cannot assume this API exists	May require compatibility libraries or providers

The Java 25 documentation specifically states that KDF objects are not generally thread-safe and that unsupported algorithms/providers can cause exceptions. ([Oracle Docs](#))

5. When to Use

Situation	Use KDF API?	Why
Deriving an AES key from shared secret material	✓ Yes	Classic KDF use case
TLS-style key schedule	✓ Yes	KDFs are fundamental to modern protocol key schedules
Deriving multiple keys from one master secret	✓ Yes	Provides cryptographic separation
Deriving keys after Diffie-Hellman/ECDH	✓ Yes	Shared secrets commonly need KDF processing
HPKE implementation	✓ Yes	KDF is part of HPKE key scheduling
KEM-based cryptography	✓ Yes	KDF can transform shared secret material into usable keys
Secure session-key generation	✓ Yes	Allows context-specific session keys
Multi-purpose master key	✓ Yes	Different info/context values can produce distinct keys
HSM/protected-key workflows	✓ Yes	Provider architecture can support non-extractable key material
Protocol-specific derived data	✓ Yes	deriveData() is intended for raw keying material

6. When Not to Use

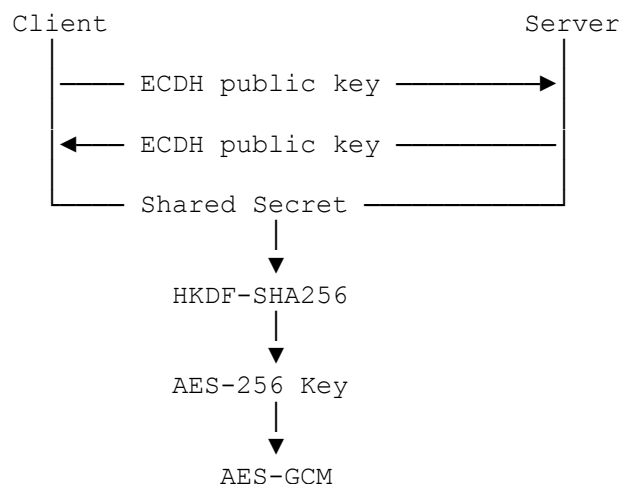
Situation	Don't use KDF API?	Better choice
Storing user passwords	✗	Argon2id, scrypt, bcrypt, or PBKDF2 depending on requirements
Encrypting data directly	✗	AES-GCM or ChaCha20-Poly1305
Generating random keys from nothing	✗	SecureRandom / appropriate key-generation API
Digital signatures	✗	EdDSA, ECDSA, RSA signatures, etc.
Hashing ordinary	✗	SHA-256/SHA-3 as appropriate

application data		
Message authentication	✗	HMAC or an AEAD mode
Key storage	✗	KMS/HSM/keystore mechanisms
User authentication	✗	An authentication protocol/identity system
Password verification	✗	Password-specific password hashing/KDF
Simple non-secret transformation	✗	Ordinary encoding/transformation

Important distinction: KDF and password KDF are related concepts, but you should not automatically substitute HKDF for a password-hashing/password-hardening algorithm.

7. Real-Time Use Case 1 — ECDH Shared Secret → AES Key

A common architecture is:



Example:

```
KDF kdf = KDF.getInstance("HKDF-SHA256");
```

```
HKDFParameterSpec spec =
    HKDFParameterSpec.ofExtract()
        .addIKM(sharedSecret)
        .addSalt(sessionSalt)
```

```

    .thenExpand(
      "client-to-server".getBytes(),
      32
    );

```

```

SecretKey aesKey =
    kdf.deriveKey("AES", spec);

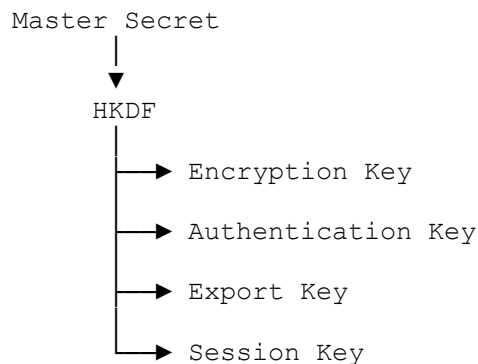
```

Why?

The ECDH shared secret should generally not simply be used directly as an AES key. A KDF can extract and expand the shared secret into key material with a defined context.

8. Real-Time Use Case 2 — One Master Secret → Multiple Keys

Suppose a service has:



Different context values can distinguish the outputs:

```

"encryption"
"authentication"
"session"
"export"

```

This is known as **key separation**.

Instead of:

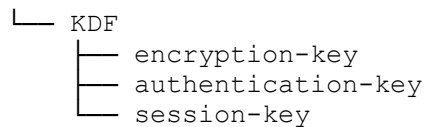
masterKey → same key everywhere

you can use:

```

masterKey
|

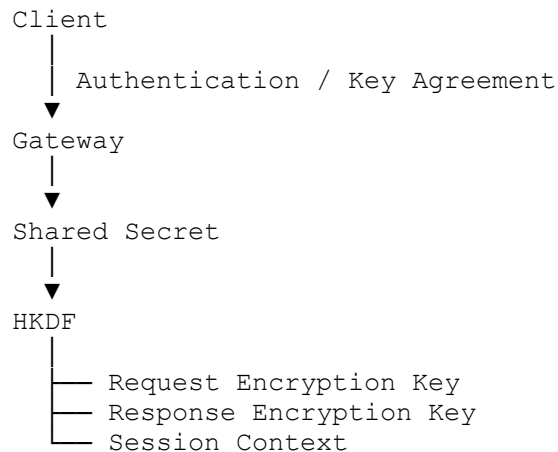
```



This is particularly useful in large distributed systems.

9. Real-Time Use Case 3 — Secure Session Establishment

For an API gateway:



Example:

```
KDF kdf = KDF.getInstance("HKDF-SHA256");

HKDFParameterSpec spec =
    HKDFParameterSpec.ofExtract()
        .addIKM(sharedSecret)
        .thenExpand(
            "api-session-v1".getBytes(),
            32
        );

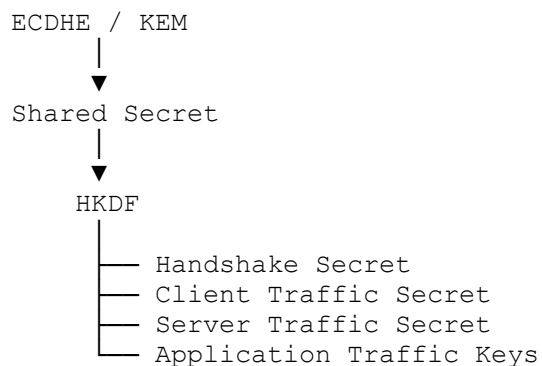
SecretKey sessionKey =
    kdf.deriveKey("AES", spec);
```

The context string ensures that the derived key has a defined purpose.

10. Real-Time Use Case 4 — TLS/Modern Protocols

KDFs are fundamental to modern secure protocols.

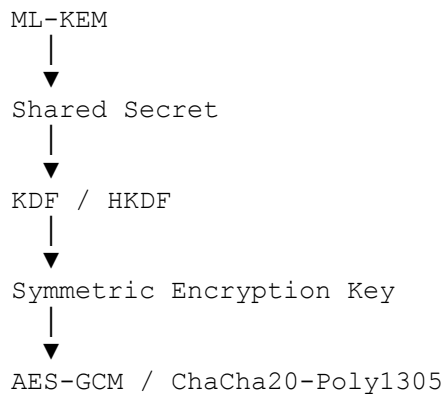
A simplified TLS-style architecture:



JEP 510 specifically identifies TLS 1.3, HPKE and KEM-related implementations as important KDF use cases. ([OpenJDK Bug System](#))

11. Real-Time Use Case 5 — Key Encapsulation / Post-Quantum Cryptography

Modern systems may use:



This separates the asymmetric/key-establishment operation from the symmetric encryption key.

JEP 510 explicitly mentions KEM implementations such as ML-KEM and protocols such as HPKE. ([OpenJDK Bug System](#))

12. KDF vs Common Alternatives

Requirement	KDF API	MessageDigest	Mac/HMAC	SecureRandom	Password
-------------	---------	---------------	----------	--------------	----------

					KDF
Derive cryptographic keys	★★★★★	★★	★★★	★	★★★★★
HKDF	✓	✗	Manual implementation	✗	✗
Generate random keys	✗	✗	✗	✓	✗
Hash data	✗	✓	✗	✗	✗
Message authentication	✗	✗	✓	✗	✗
Password hardening	✗	✗	✗	✗	✓
Multiple purpose-specific keys	✓	? Manual	? Manual	✗	?
Protocol key schedule	✓	✗	? Manual	✗	✗

13. Alternate Features / Technologies

There isn't one single "replacement" for the KDF API. The appropriate alternative depends on what you're trying to accomplish.

Requirement	Alternative	When preferable
Password-based key derivation	PBKDF2	When compatibility with existing Java/security standards is required
Password hashing	Argon2id	New password-storage systems where Argon2id is available and appropriate
Password hashing	scrypt	Memory-hard password protection
Password hashing	bcrypt	Legacy systems with bcrypt compatibility

Generic cryptographic MAC	HMAC (Mac)	When you need authentication rather than standardized KDF operations
Random key generation	SecureRandom + KeyGenerator	When you need a fresh random key rather than a deterministic derived key
Existing ECDH key agreement	KeyAgreement	When the requirement is establishing a shared secret, not deriving application keys
KEM	KEM API	When the primary requirement is key encapsulation rather than key derivation
Third-party KDF	Bouncy Castle KDF APIs	When supporting algorithms/providers beyond the JDK's built-in capabilities or older Java versions

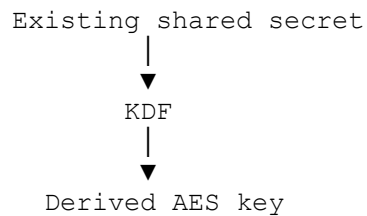
The Java 25 platform also has a separate KEM API; KEM and KDF solve different stages of a cryptographic workflow. ([Oracle Docs](#))

14. KDF vs KeyGenerator

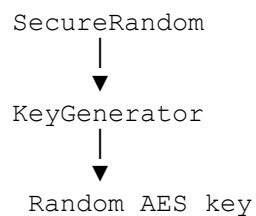
Feature	KDF	KeyGenerator
Primary purpose	Derive key from existing secret material	Generate a new random key
Input	Existing keying material	Randomness
Deterministic	Usually yes for same inputs	No
Uses shared secret	✓	✗
Uses salt/context	✓	✗
Session-key derivation	✓ Excellent	🔍 Different purpose
Fresh random AES key	✗	✓ Excellent
Protocol key schedule	✓	✗

Key separation	✓	✗

Example:



versus:



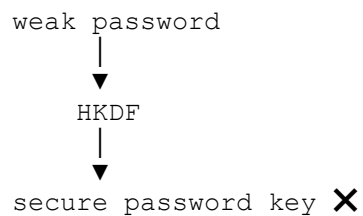
15. Important Security Rule

A KDF does **not** magically make bad key material secure.

For example:

```
KDF kdf = KDF.getInstance("HKDF-SHA256");
```

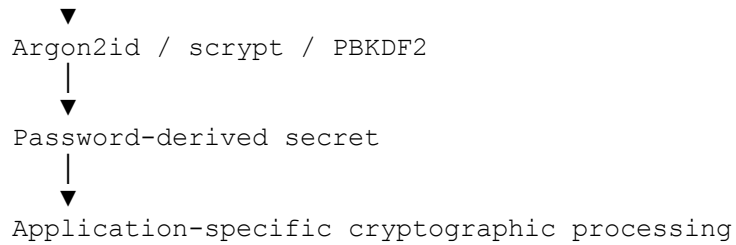
doesn't mean this is automatically secure:



For passwords, use an appropriate password-hardening scheme.

A more appropriate architecture is:





16. Overall Assessment

Category	Rating	Assessment
Security usefulness	★★★★★	Excellent for proper key derivation
API design	★★★★★	Clean JCA-style API
HKDF support	★★★★★	Strong modern use case
Protocol integration	★★★★★	Very useful for modern cryptographic protocols
Ease of use	★★★★	Reasonable after understanding KDF concepts
Password handling	★★	Not its primary purpose
Random key generation	★	Use SecureRandom/KeyGenerator
Key separation	★★★★★	Excellent
Provider integration	★★★★★	Strong JCA architecture
Legacy Java compatibility	★★	Requires Java 25 API or alternative provider/library

17. Final Decision Table

Real-world requirement	Recommendation
ECDH shared secret → AES key	✓ Use KDF API
ML-KEM shared secret → symmetric key	✓ Use KDF API

TLS/HPKE-style key schedule	✓ Use KDF API
Master secret → multiple independent keys	✓ Use KDF API
Session-key derivation	✓ Use KDF API
HKDF implementation	✓ Use KDF API
Generate random AES key	✗ Use KeyGenerator
Generate random bytes	✗ Use SecureRandom
Store user passwords	✗ Use Argon2id/scrypt/PBKDF2 as appropriate
Encrypt files	✗ Use AES-GCM/ChaCha20-Poly1305
Create digital signatures	✗ Use signature APIs
Authenticate messages	✗ Use MAC/AEAD
Store/manage keys	✗ Use a keystore/KMS/HSM

Bottom line

Java 25's Key Derivation Function API is the preferred standard JDK mechanism when you already have secret/keying material and need to derive cryptographically separate keys or keying data from it. It is particularly valuable for **HKDF, ECDH/KEM-based protocols, TLS 1.3/HPKE-style key schedules, session keys, and key separation.** Java 25 provides KDF, KDFParameters, and HKDF-specific APIs directly in the JDK. ([Oracle Docs](#))

Simple rule:

Need a NEW random key? → SecureRandom / KeyGenerator
 Have a SECRET and derive? → KDF
 Have a PASSWORD? → Password KDF (Argon2id/scrypt/PBKDF2)
 Need to ENCRYPT? → AES-GCM / ChaCha20-Poly1305
 Need KEY AGREEMENT? → ECDH / KeyAgreement / KEM